

Threshold Key Encapsulation Mechanisms via MPC: ML-KEM Compatibility and Optimisation

The-Anh Ta, Dongxi Liu, Sid Chi-Kin Chau, Jiafan Wang
CSIRO, Australia

Abstract

There is currently a strong interest in designing standard-compatible threshold cryptographic primitives, a trend bolstered by NIST's Call for Multi-Party Threshold Schemes (NIST IR 8214C). While Multi-Party Computation (MPC) can be applied to the construction of threshold Key Encapsulation Mechanism (KEM) from NIST standard ML-KEM scheme to preserve the compatibility with ML-KEM, such a direct construction yields a high number of threshold decapsulation rounds as many as 384 for MK-KEM-512. In this paper, we optimise the round complexity of MPC-based threshold ML-KEM decapsulation. Our first construction (Scheme A) uses masked predicates, parallel coin expansion and global scheduling methods to improve the round complexity of MPC-based ML-KEM decapsulation, while maintaining ML-KEM encapsulation, public key and ciphertext formats. For ML-KEM-512, 768 and 1024, our construction improves the round complexity to 145, 218, and 290, respectively. We also introduce a variant (Scheme B) that uses a MPC-friendly variant of Kyber KEM and KangarooTwelve hash to further improve the round complexity to 35 at the security level of ML-KEM-512, without full ML-KEM compatibility. In the semi-honest MPC model, we prove Scheme A security by exact composition with ML-KEM, and Scheme B IND-CCA security in the QROM from Module-LWE and negligible decryption failure.

CCS Concepts

• **Security and privacy** → **Public key (asymmetric) techniques**; *Key management*.

Keywords

Threshold cryptography, key encapsulation mechanisms, ML-KEM, post-quantum cryptography, multi-party computation

1 Introduction

A *Key Encapsulation Mechanism* (KEM) is an asymmetric cryptographic primitive used to securely establish a shared symmetric key over an insecure channel. Under a KEM scheme, a sender encapsulates a secret session key into encrypted data using the receiver's public key, which can then be decapsulated by the receiver's private key to recover the session key for subsequent confidential communications. KEMs have become a dominant standard in post-quantum cryptography (PQC) because primitive post-quantum public-key encryption schemes, such as those built on lattice or code-based problems, suffer from high overhead and ciphertext expansion factors when encrypting arbitrary plaintext data directly.

NIST has standardised post-quantum KEM schemes, enabling seamless deployment within established networking protocols, including TLS, SSH, and VPNs. Standardised post-quantum KEMs include Kyber [5] as ML-KEM in FIPS 203 [26] and up-coming code-based scheme HQC-KEM, as expected in FIPS 207 [1]. Particularly, ML-KEM demonstrates strong operational efficiency, low ciphertext expansion, and readiness for hybrid key schemes. Consequently, ML-KEM has gained rapid adoption across mainstream web browsers (e.g., Chrome, Firefox), reverse proxy services (e.g., Cloudflare), and open-source cryptographic software stacks (e.g., OpenSSL, BoringSSL).

Conventional KEMs rely on a single private key stored in one location, creating a high-value single point of failure. *Threshold cryptography* [14] eliminates this vulnerability by splitting the decapsulation key across a network of independent nodes. Because no single party holds the complete key, establishing a shared secret requires a quorum of nodes to cooperate. As organisations prepare for the post-quantum era, developing robust, post-quantum threshold KEMs is essential for securing high-value digital infrastructure.

Recently, NIST initiated a Call for Multi-Party Threshold Schemes (NIST IR 8214C [8]), specifically inviting threshold implementations possibly compatible with its approved standards. In particular, a *threshold KEM* (TKEM) scheme enables multiple parties to collaboratively decapsulate messages without reconstructing the private key, enhancing fault tolerance, trustworthiness, and privacy.

To illustrate a practical application of TKEM, we consider a use case of *reverse proxies* integrated with *keyless SSL* (e.g., Cloudflare [11]). In a standard keyless SSL deployment, a reverse proxy terminates TLS handshakes and mitigates DDoS attacks on behalf of a customer backend server. To avoid exposing the customer's private key to the proxy, the proxy forwards the client's encrypted session payload to a single customer-managed key server housing the private key. Under a (t, N) -TKEM scheme, the customer's private decapsulation key is split into N secret shares distributed across the proxy and multiple independent key servers. Establishing the TLS session requires a quorum of t nodes to collaboratively generate partial decapsulation shares. This ensures the shared secret can be derived without the proxy or any single entity ever gaining full possession of the master private key¹.

In this work, we address the two fundamental research questions that are raised in the Scope of the Call of NIST IR 8214C [8]:

- (1) *How can existing NIST-approved primitives (e.g., ML-KEM) be converted into multi-party threshold protocols without altering the baseline standard?* - Referred in Class N.

¹Typical TLS connections require both a KEM for confidential data transmission and a signature scheme for validating source authenticity. While post-quantum TLS 1.3 relies on ML-KEM alongside a separate signature scheme like ML-DSA, recent efforts like KEMTLS unify confidential data transmission and source authentication using ML-KEM alone. There are ongoing efforts to standardise KEM-based TLS authentication within the IETF [32].

- (2) *How can non-standardised but threshold-friendly approaches be harnessed to enhance conventional standard-compatible constructions?* - Referred in Class S.

To address the first question in the context of KEMs, we first construct a threshold variant of ML-KEM. A natural starting point is to apply general Multi-Party Computation (MPC) techniques to the decapsulation circuit. However, a naive MPC formulation introduces significant operational bottlenecks. Because post-quantum KEMs (including ML-KEM) incorporate symmetric hash functions and implicit rejection mechanisms via the Fujisaki-Okamoto (FO) transform [15, 17] during decapsulation, evaluating these non-linear components inside MPC circuits leads to high round complexity and substantial communication overhead. Therefore, we optimise the efficiency of MPC-based TKEMs while preserving the exact standardised ML-KEM decapsulation circuit (FIPS 203).

To address the second question, we further relax full ML-KEM compatibility by introducing MPC-friendly components to ML-KEM, significantly reducing the round complexity and communication overhead.

1.1 Design Overview

We first explain the high-level design of our TKEMs:

► **Targeted Use Case.** We focus on the setting of *pre-owned private key*, where a user or organisation has already generated a public-private key pair and seeks to distribute secret shares of the existing private key among multiple independent custodians. This model directly aligns with real-world architectures such as reverse proxies integrated with keyless SSL. Because our TKEMs are tailored specifically for pre-owned private key, it avoids the expensive Distributed Key Generation (DKG) required by fully distributed setups, making it significantly more practical for enterprise deployment.

► **Scheme A: Fully ML-KEM compatible TKEM.** We first decompose the ML-KEM-512 decapsulation circuit into fundamental MPC operations, explicitly accounting for non-linear components such as the secret-input hash and fallback hash (arising from the FO transform). This decomposition yields a baseline MPC implementation that serves as our benchmark, requiring approximately 384 online interactive rounds prior to final key reconstruction.

To design a round-efficient scheme, we introduce three optimisation techniques. First, we use fresh correlated masks to reduce decoding and validation to the interval predicates that ML-KEM-512 actually invokes. This reduces the number of rounds from 384 to 360. Second, we arrange the five independent coin-expansion calls by parallel computation, reducing the number of rounds from 360 to 240. Third, we rewire the decapsulation circuit so that the ciphertext validation and implicit rejection branches, of depth 95 and 144 respectively, can be performed concurrently. The whole MPC protocol has only 145 rounds, with an additional batched opening to reconstruct the 256-bit session key.

Crucially, none of these optimisation techniques modifies the external ML-KEM interface. Scheme A uses standard ML-KEM public keys and ciphertexts, leaving the native encapsulation algorithm entirely unchanged. Furthermore, it reconstructs the exact same session key as standard ML-KEM decapsulation for every input ciphertext, including invalid ciphertexts that trigger the implicit rejection mechanism. We will elaborate its compatibility in Section 6.

► **Scheme B: More efficient MPC-friendly variant.** Scheme A achieves full compatibility with standardised ML-KEM-512, albeit at the cost of evaluating a 144-round secret fallback hash; further efficiency improvements can only be realised by modifying internal ML-KEM components. Therefore, we introduce Scheme B based on a MPC-friendly variant of Kyber. This variant retains an underlying module-lattice structure closely aligned with ML-KEM, but utilises uncompressed canonical ciphertext coefficients alongside the KangarooTwelve hash (specifically, the domain separated RFC 9861 KT128 function) for matrix expansion and key derivation. These design choices yield several key MPC-friendly advantages: the five secret expansion queries fit within individual KT128 leaf evaluations that execute entirely in parallel, ciphertext verification reduces to an exact equality check, and the valid and fallback key derivations proceed concurrently alongside ciphertext validation.

By relaxing full ML-KEM-512 compatibility, the resulting threshold MPC-based decapsulation protocol achieves a minimal round complexity of just 35 rounds. In this relaxed variant, public keys and ciphertexts are 2,304 bytes and 1,152 bytes, respectively.

1.2 Security Overview

Our TKEMs are presented in semi-honest security model. The security proofs for Scheme A and B are established in the static, semi-honest MPC model with honest, trusted preprocessing, tolerating up to $t - 1$ corrupted parties. We do not consider active adversaries, adaptive corruptions, or guaranteed output delivery. We will also discuss their extensions to the malicious security model and fully decentralised setting in the subsequent section.

Our core MPC-based protocols rely on Shamir secret sharing, Beaver multiplication triples, correlated randomness, and trusted offline preprocessing. Against any static semi-honest adversary corrupting fewer than t parties, the protocol transcript is perfectly simulatable. Specifically, sub-threshold Shamir shares are statistically independent of the underlying secret, and every value revealed during multiplication, secret conversion, or masked predicate evaluation is perfectly masked by one-time preprocessing randomness.

Scheme A strictly inherits the IND-CCA security of ML-KEM, requiring no additional computational assumptions beyond those of the standard model within this MPC setting.

For Scheme B, we first establish the IND-CPA security of the underlying PKE under the Module-LWE assumption, bounding its worst-case decryption failure probability by $\delta_B < 2^{-227.5}$. By modeling KT128's evaluations as quantum random oracles, we apply the Kyber-style QROM reduction to Scheme B following Maram and Xagawa [23]. Its overall threshold security then follows by composition with the underlying, perfectly passive MPC protocol.

Section 8 presents the security theorems of our TKEMs, while the complete proofs are provided in Appendix B.

1.3 Contributions

► **Standard-Compatible KEM from MPC.** In Scheme A, we design fully ML-KEM-compatible TKEMs for ML-KEM-512, 768, and 1024, achieving 145, 217, and 289 online rounds, respectively. This improves the interactive round complexity of MPC-based threshold ML-KEM compared to a direct application of MPC techniques.

► **Efficient MPC-Friendly Variant.** In Scheme B, we design an MPC-friendly variant of Kyber. While retaining a module-lattice structure closely aligned with ML-KEM, it drastically reduces threshold decapsulation latency to just 35 online rounds at the security level of ML-KEM-512.

► **Complete Security Analysis.** We prove that Scheme A strictly inherits the IND-CCA security of ML-KEM without introducing additional computational assumptions, while achieving perfect passive MPC security. For Scheme B, we provide an IND-CPA reduction of its underlying PKE to the Module-LWE problem, conduct a tight decryption failure analysis, and prove QROM IND-CCA security alongside threshold security in the same MPC model.

► **Implementation and Evaluation.** We implemented our TKEMs and conducted extensive performance evaluation. For the (2, 3)-TKEM setting, the communication sizes of each party are 0.29MB for Scheme A-512 and 0.08MB for Scheme B-512. We release open-source implementation of our TKEMs.

Organisation. Section 2 reviews the related work. Section 4 provides a technical overview of our TKEM constructions. Section 3 factors ML-KEM decapsulation into the operations for applying MPC. Section 5 provides their MPC protocols for realisation of these operations. Sections 6-7 present our TKEM schemes. Section 8 presents the main security results. Section 9 describes implementation and evaluation. The appendices provide additional details on MPC protocols, full security proofs, extensions of our scheme A to ML-KEM-768 and 1024, and further discussion on extensions and deployment scenarios.

2 Related Work

► **Threshold Lattice-based PKE.** There have been several proposals on lattice based PKE schemes with threshold decryption. However, these proposals do not support threshold KEM. Early work of Bendlin and Damgard [3] constructed threshold decryption for a Regev type cryptosystem [27]. Boneh et al. [4] showed how threshold fully homomorphic encryption can be used for blackbox construction of threshold cryptosystems. Micciancio and Suhl [24] constructed simulation-secure lattice threshold PKE with a polynomially bounded modulus. Recently, Hwang et al. [18] constructed threshold Regev PKE which is secure from standard MLWE with several desirable features.

Compared to ordinary PKE, security models for threshold PKE are more subtle. Recently, Brzuska et al. [9] provide a systematic treatment of IND versus simulation based confidentiality, CPA versus CCA attacks, and static versus adaptive corruptions. They show that security proved only at maximal corruption threshold need not automatically imply security for fewer corruptions when challenge partial decryptions are available. A recent lattice based threshold PKE scheme [10] addresses these new security notions. Given the similarity between PKE and KEM, one can expect that similar security analysis may apply to KEM.

Previous work has also applied MPC techniques for threshold decryption of lattice based PKE. Kraitsberg et al. [20] thresholdised a Ring-LWE based CCA encryption scheme using various MPC techniques. They also reported early MPC cost for SHA3 evaluation.

Gladius [12] develops an IND-CCA secure hybrid public key encryption scheme with distributed decryption. Its design modifies the KEM-DEM paradigm around an LWR and Module-LWR public key component so that the validity checks and key release can be implemented efficiently with generic MPC. Its distributed decryption instantiation use Shamir sharing based MPC with reported 136, 491 rounds for 1-out-of-3 threshold setting.

► **TKEM Candidate: Amber.** A line of work constructs threshold KEMs natively to avoid generic MPC evaluation of a standard KEM. Lapiha and Prest [21] introduced a lattice based TKEM from the BCHK+ transform by combining a threshold IBE with a one-time signature (OTS) to obtain threshold friendly decapsulation. Boudgoust et al. [7] improved this construction by reducing ciphertext size and introducing verifiable extraction shares against malicious participants. The subsequent Amber proposal [6] to the NIST Call builds upon these works, and also targets active adversary security. Stronger security properties of Amber depends on its variants. The Shamir sharing based construction supports adaptive corruptions, whereas their Vandermonde sharing based variant provides extraction share robustness under static corruptions.

The TKEM schemes in the Amber family possess a different design point from ours. They redesign the cryptosystem for native, round efficient threshold decapsulation and stronger malicious adversary guarantees, whereas scheme A prioritises *full ML-KEM interoperability*, and our TKEMs use perfectly private MPC against static semi-honest corruptions under trusted preprocessing.

The comparison between Amber and our schemes is presented in Table 1. We use the 128-bit parameter set from [7], since the current Amber document does not yet provide final byte level parameters. This parameter set supports $t \leq N \leq 32$. As communication cost is not reported for Amber, we derive from [7] an estimate of approximately 50KB per active party from the 3-round protocol, excluding transport and broadcast replication.

3 Preliminaries

We recall the notation for ML-KEM as specified in FIPS 203 [26] which is used throughout the paper. We work specifically with ML-KEM-512 and organise its computational components so that each operation acting on secret shared data has an explicit name. Later sections will describe MPC protocol for each operation. Our scheme A will assemble those into a threshold decapsulation protocol. Scheme A also admits variants for ML-KEM-768 and 1024 which are obtained by changing the parameters. These schemes are treated separately in the Appendix.

3.1 ML-KEM-512 Notations

Let $\mathbb{B} = \{0, \dots, 255\}$ denote the set of bytes. $\mathbb{B}^\ell$ is the set of ℓ -byte arrays. Concatenation is written as $\parallel$. For ML-KEM, the degree is $n = 256$, modulus is $q = 3329$, and the ring is $R_q = \mathbb{Z}_q[X]/(X^n + 1)$.

Vectors and matrices over R_q are written in bold. The number theoretic transform NTT and its inverse NTT^{-1} are linear map over $\mathbb{Z}_q$. A hat denotes the NTT representation, for example, $\hat{s} = \text{NTT}(s)$. We write $\circ$ for multiplication operation in the NTT representation, which is extended naturally to matrix-vector and vector inner products. Thus, $\hat{A}^T \circ \hat{y}$ and $\hat{s}^T \circ \text{NTT}(u)$ denote the same ring products used in FIPS 203, only evaluated in the NTT domain.

Table 1: Qualitative comparison of lattice-based TKEM schemes.

Construction	Primitives/ Baseline Standard	Max N Supported	Security				Efficiency			
			PQC Security	Oracle Model	Threshold Security	Adversary Model	Encapsulation Key Length	Ciphertext Length	Round	Comm. Per Party (t, N) = (16, 32)
Amber [6, 7, 21]	BCHK+ from lattice TIBE, OTS	32	128-bit	ROM	Active computational	Adaptive, robust	8, 224B	31, 931B [†]	3	50KB
Scheme A-512	ML-KEM-512	3328	NIST PQC Category 1	QROM	Passive info. theoretic	Static, semi-honest	800B	768B	145	862KB
Scheme A-768	ML-KEM-768	3328	NIST PQC Category 3	QROM	Passive info. theoretic	Static, semi-honest	1, 184B	1, 088B	217	1, 050KB
Scheme A-1024	ML-KEM-1024	3328	NIST PQC Category 5	QROM	Passive info. theoretic	Static, semi-honest	1, 568B	1, 568B	289	1, 350KB
Scheme B-512	Module-LPR, KT128, FO transform	3328	NIST PQC Category 1	QROM	Passive info. theoretic	Static, semi-honest	2, 304B	1, 152B	35	234KB

[†] For Amber using Vandermonde sharing, an additional ≈ 70 MB public dictionary is required for $(t, N) = (16, 32)$.

[‡] The NIST PQC category is defined in order of increasing strength. As per its definition, Category 5 > Category 3 > Category 1 $\approx$ 128-bit AES.

ML-KEM-512 [26] used the following fixed parameters: $k = 2$, $\eta_1 = 3$, $\eta_2 = 2$, $d_u = 10$, $d_v = 4$.

3.1.1 Compression and Message Encoding. For $1 \leq d < 12$, ML-KEM defines coefficient compression and decompression by nearest integer rounding $\lfloor x \rfloor$ as

$$\text{Compress}_d(x) = \left\lfloor \frac{2^d x}{q} \right\rfloor \pmod{2^d}, \quad (1)$$

$$\text{Decompress}_d(y) = \left\lfloor \frac{qy}{2^d} \right\rfloor. \quad (2)$$

Functions ByteEncode_d and ByteDecode_d serialise and parse arrays of 256 coefficients, using d bits per coefficient. We apply these operations coefficient-wise to polynomials and vectors.

A 32-byte message encodes one bit into each polynomial coefficient. The message encoding and decoding functions are defined as

$$\text{EncodeMsg}(m) := \text{Decompress}_1(\text{ByteDecode}_1(m)), \quad (3)$$

$$\text{DecodeMsg}(w) := \text{ByteEncode}_1(\text{Compress}_1(w)). \quad (4)$$

For $x \in \{0, \dots, q-1\}$, $\text{Compress}_1(x) = 1$ if and only if $833 \leq x \leq 2496$. This will allow replacing a generic arithmetic to Boolean conversion by a masked interval predicate.

3.1.2 Hash and Pseudorandom Functions. ML-KEM-512 uses the following functions based on the SHA3 family [25, 26]

$$H(x) := \text{SHA3-256}(x), \quad (5)$$

$$G(x) := \text{SHA3-512}(x), \quad (6)$$

$$J(x) := \text{SHAKE256}(x, 32), \quad (7)$$

$$\text{PRF}_\eta(s, b) := \text{SHAKE256}(s \| b, 64\eta), \quad \eta \in \{2, 3\}, \quad (8)$$

where lengths in Eqs. (7)–(8) are written in bytes. The value $G(x)$ is parsed as two 32-byte strings. The nonce b is exactly one byte.

3.2 MPC-Oriented Pseudocode

Algorithm 1 presents the MPC-oriented description of ML-KEM-512 used in this paper. For simplicity, we assume inputs have passed the public API checks required by FIPS 203 specification, which are performed by an additional decapsulation-orthogonal protocol. Our solution realise decapsulation computation after these checks.

Algorithm 1 MPC-oriented pseudocode of ML-KEM-512.

```

1: procedure ML-KEM-512.KEYGEN
2:    $d \xleftarrow{\$} \mathbb{B}^{32}$ 
3:    $z \xleftarrow{\$} \mathbb{B}^{32}$ 
4:    $(ek_{\text{PKE}}, dk_{\text{PKE}}) \leftarrow \text{K-PKE.KEYGEN}(d)$ 
5:    $h \leftarrow H(ek_{\text{PKE}})$ 
6:    $ek \leftarrow ek_{\text{PKE}}$ 
7:    $dk \leftarrow dk_{\text{PKE}} \| ek \| h \| z$ 
8:   return  $(ek, dk)$ 
9: end procedure

10: procedure ML-KEM-512.ENCAPS( $ek$ )
11:    $m \xleftarrow{\$} \mathbb{B}^{32}$ 
12:    $h \leftarrow H(ek)$ 
13:    $(K, r) \leftarrow G(m \| h)$ 
14:    $(y, e_1, e_2) \leftarrow \text{EXPANDCOINS}(r)$ 
15:    $(u, v) \leftarrow \text{REENCLIN}(ek, m, y, e_1, e_2)$ 
16:    $c \leftarrow \text{PACKCT}(u, v)$ 
17:   return  $(K, c)$ 
18: end procedure

19: procedure ML-KEM-512.DECAPS( $dk, c$ )
20:    $dk_{\text{PKE}} \| ek \| h \| z \leftarrow dk$ 
21:    $w \leftarrow \text{DECLIN}(dk_{\text{PKE}}, c)$ 
22:    $m' \leftarrow \text{DECODEMSG}(w)$ 
23:    $(K', r') \leftarrow G(m' \| h)$ 
24:    $K_\perp \leftarrow J(z \| c)$ 
25:    $(y', e'_1, e'_2) \leftarrow \text{EXPANDCOINS}(r')$ 
26:    $(u', v') \leftarrow \text{REENCLIN}(ek, m', y', e'_1, e'_2)$ 
27:    $b \leftarrow \text{COMPEQ}((u', v'), c)$ 
28:    $K \leftarrow \text{SELECT}(b, K', K_\perp)$ 
29:   return  $K$ 
30: end procedure

```

The pseudocode shows two branches of the decapsulation circuit. The ciphertext validation branch recovers m' , derives (K', r') , reencrypts m' , then obtains the secret validity bit b . The fallback branch computes $K_\perp = J(z \| c)$. The branches meet only at Select. ML-KEM use implicit rejection which hides the ciphertext validation check result. Next, we describe the helper functions used in Algorithm 1.

3.2.1 Public Key Expansion and Key Generation. For a public 32-byte seed ρ , we define a 2×2 matrix $\hat{\mathbf{A}} = \text{ExpandA}(\rho)$ using

$$\text{ExpandA}(\rho)[i, j] := \text{SampleNTT}(\rho \| j \| i), \quad i, j \in \{0, 1\}, \quad (9)$$

where i and j are encoded as single bytes, SampleNTT is the rejection sampling procedure, and the order $\rho \| j \| i$ is specified in FIPS 203. Since ρ is part of the public encryption key, $\hat{\mathbf{A}}$ is public.

For $k = 2$, Kyber PKE key generation in ML-KEM is presented in Algorithm 4. The byte $\langle 2 \rangle_8$ is the module dimension which provides the parameter set domain separation. The input for randomness expansion is $G(d\|\langle 2 \rangle_8)$.

3.2.2 Ciphertext Parsing and Linear Decryption Step. An ML-KEM-512 ciphertext is parsed as $c = c_u \| c_v$. We define

$$\mathbf{u}_c := \text{ByteDecode}_{d_u}(c_u), \quad \mathbf{v}_c := \text{ByteDecode}_{d_v}(c_v), \quad (10)$$

$$\mathbf{u} := \text{Decompress}_{d_u}(\mathbf{u}_c), \quad \mathbf{v} := \text{Decompress}_{d_v}(\mathbf{v}_c). \quad (11)$$

Since c is public, all quantities in Eqs. (10)–(11) are also public. For $dk_{\text{PKE}} = \text{ByteEncode}_{12}(\hat{\mathbf{s}})$, we define

$$\text{DecLin}(dk_{\text{PKE}}, c) := v - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})), \quad (12)$$

from which the PKE decryption step is factored as

$$\text{K-PKE.Decrypt}(dk_{\text{PKE}}, c) = \text{DecodeMsg}(\text{DecLin}(dk_{\text{PKE}}, c)). \quad (13)$$

3.2.3 Expansion of Encryption Coins. Kyber PKE encryption uses five domain separated PRF outputs derived from its 32-byte randomness r , which is collectively denoted as the operation

$$\text{ExpandCoins}(r) := (\mathbf{y}, \mathbf{e}_1, \mathbf{e}_2), \quad (14)$$

where

$$\mathbf{y}[i] = \text{SamplePolyCBD}_3(\text{PRF}_3(r, \langle i \rangle_8)), \quad i \in \{0, 1\}, \quad (15)$$

$$\mathbf{e}_1[i] = \text{SamplePolyCBD}_2(\text{PRF}_2(r, \langle 2 + i \rangle_8)), \quad i \in \{0, 1\}, \quad (16)$$

$$\mathbf{e}_2 = \text{SamplePolyCBD}_2(\text{PRF}_2(r, \langle 4 \rangle_8)), \quad (17)$$

where SamplePolyCBD is the CBD sampler used by ML-KEM to convert pseudorandom bytes into a small noise polynomial. Once r is available, these five calls have no data dependencies on one another which allows their MPC evaluations to be scheduled in parallel.

3.2.4 Reencryption. The encapsulation key is parsed as

$$\hat{\mathbf{t}} := \text{ByteDecode}_{12}(ek[0 : 768]), \quad \rho := ek[768 : 800]. \quad (18)$$

For given $(m, \mathbf{y}, \mathbf{e}_1, \mathbf{e}_2)$, one defines

$$\hat{\mathbf{y}} := \text{NTT}(\mathbf{y}), \quad (19)$$

$$\mathbf{u}' := \text{NTT}^{-1}(\hat{\mathbf{A}}^T \circ \hat{\mathbf{y}}) + \mathbf{e}_1, \quad (20)$$

$$\mathbf{v}' := \text{NTT}^{-1}(\hat{\mathbf{t}}^T \circ \hat{\mathbf{y}}) + \mathbf{e}_2 + \text{EncodeMsg}(m). \quad (21)$$

We write

$$\text{ReEncLin}(ek, m, \mathbf{y}, \mathbf{e}_1, \mathbf{e}_2) := (\mathbf{u}', \mathbf{v}'). \quad (22)$$

Once the secret values $m, \mathbf{y}, \mathbf{e}_1, \mathbf{e}_2$ have suitable arithmetic representations, every ring multiplication in Eqs. (20)–(21) has one public operand either $\hat{\mathbf{A}}$ or $\hat{\mathbf{t}}$. Therefore, they are local linear operation on Shamir secret shares.

3.2.5 Ciphertext Serialisation and Compressed Equality Check. The standard Kyber PKE ciphertext serialisation is

$$\begin{aligned} \text{PackCT}(\mathbf{u}', \mathbf{v}') &:= \text{ByteEncode}_{d_u}(\text{Compress}_{d_u}(\mathbf{u}')) \\ &\parallel \text{ByteEncode}_{d_v}(\text{Compress}_{d_v}(\mathbf{v}')). \end{aligned} \quad (23)$$

During decapsulation, we need to check whether reencrypted ciphertext matches the public one, which gives the predicate

$$\text{CompEq}((\mathbf{u}', \mathbf{v}'), c) := \mathbf{1}[\text{PackCT}(\mathbf{u}', \mathbf{v}') = c]. \quad (24)$$

3.2.6 Implicit Rejection and Secret Selection. For a ciphertext validity check bit $b \in \{0, 1\}$, a candidate key $K' \in \mathbb{B}^{32}$, and a fallback key $K_\perp \in \mathbb{B}^{32}$, we define

$$\text{Select}(b, K', K_\perp) := \begin{cases} K', & b = 1, \\ K_\perp, & b = 0, \end{cases} \quad (25)$$

which can be equivalently written as bitwise operation over $\mathbb{F}_2$ as

$$\text{Select}(b, K', K_\perp) = K_\perp \oplus b(K' \oplus K_\perp). \quad (26)$$

3.2.7 Secret Values Under Threshold Decapsulation. For every validated encapsulation and decapsulation keys, and ciphertext, Algorithm 1 returns exactly the same outputs as the corresponding standardised ML-KEM-512 algorithm, assuming that the same random bytes are used by key generation and encapsulation.

From this MPC oriented pseudocode, we can distinguish operations that are local on secret share from those requiring online MPC interactions. Since the parties hold secret shares of $\hat{\mathbf{s}}$ and $\mathbf{z}$, while $ek, h = H(ek)$ and c are public, the matrix $\hat{\mathbf{A}}$, parsed ciphertext $(\mathbf{u}_c, \mathbf{v}_c, \mathbf{u}, \mathbf{v})$ and public key $\hat{\mathbf{t}}$ are public as well. Table 4 summaries secret input and type of MPC evaluation for each operation in ML-KEM-512 decapsulation.

4 Technical Rationale

In this section, we first explain how MPC heavy components in the decapsulation circuit of ML-KEM will lead to a large number of rounds of 384 in a straightforward application of MPC for threshold decapsulation of ML-KEM-512 which we use as the baseline. Then, we present our ideas for rewiring the circuit to reduce the round complexity to 145, while keeping full compatibility with ML-KEM-512. Finally, we explain how to design an MPC friendly variant of ML-KEM to build a TKEM with 35 rounds by relaxing compatibility.

4.1 A Baseline MPC based TKEM

Let $c = (\mathbf{u}, \mathbf{v})$ be a public ML-KEM-512 ciphertext, and $\hat{\mathbf{s}}$ and $\mathbf{z}$ be the corresponding two decapsulation secrets. After public parsing and decompression, the decapsulator computes

$$\mathbf{w} = \mathbf{v} - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})), \quad (27)$$

where only $\hat{\mathbf{s}}$ is secret. Therefore, map $[\hat{\mathbf{s}}] \mapsto [\mathbf{w}]$ on shares is local in a linear secret sharing scheme. The same observation applies to reencryption once the fresh coins and recovered message have been converted to arithmetic shares.

MPC heavy operations follow after this linear step. Using the operation decomposition of ML-KEM in Section 3, the ciphertext validation branch of decapsulation is

$$\begin{aligned} [\hat{\mathbf{s}}], c &\longrightarrow [\mathbf{w}] \longrightarrow [m'] \longrightarrow G([m'] \| h) \longrightarrow ([K'], [r']) \\ &\longrightarrow \text{ExpandCoins}([r']) \longrightarrow \text{ReEncLin} \longrightarrow [b], \end{aligned} \quad (28)$$

where $b = 1$ if and only if the regenerated compressed ciphertext equals the received one. The implicit rejection in ML-KEM decapsulation defines the fallback branch

$$[z], c \longrightarrow J([z] \| c) = [K_\perp]. \quad (29)$$

The two branches meet only at the secret selection step

$$[K] = [K_\perp] + [b]([K'] - [K_\perp]), \quad (30)$$

with the equivalent XOR domain implementation for bit strings.

This factorisation of ML-KEM decapsulation circuit indicates a straightforward strategy for applying MPC for threshold decapsulation. The parties can locally perform public linear lattice operations, and apply suitable MPC protocols to evaluate the remaining operations on secret shares.

We use this approach with Shamir secret sharing as the baseline for MPC based TKEM construction for ML-KEM. The generic MPC protocols for hashing, conversion, and comparison gives the following round complexity for the case of ML-KEM-512

$$R_{\text{baseline}}^{\text{decap}} = \underbrace{18}_{\text{decode}} + \underbrace{24}_G + \underbrace{168}_{\text{PRFs}} + \underbrace{1}_{\text{B2A}} + \underbrace{28}_{\text{equality}} + \underbrace{144}_J + \underbrace{1}_{\text{mux}} = 384. \quad (31)$$

4.2 Optimisation in Scheme A

In Scheme A, we will improve the round complexity of the baseline by rescheduling the decapsulation circuit.

4.2.1 Using Exact Predicates instead of Bit Decomposition. Our first observation is that message decoding in ML-KEM does not require full bit decomposition of a secret coefficient, but only the interval membership test: $\text{Compress}_1(x) = 1$ if and only if $833 \leq x \leq 2496$.

Ciphertext validation also does not require explicit calculation of the secret shared compressed coefficients. It suffices to test whether the underlying secret shared coefficient lies in the corresponding public compression interval. Specifically, for each public compressed coefficient a , one needs to test whether the candidate coefficient lies in the public interval

$$I_{a,d} = \{x \in \mathbb{Z}_q : \text{Compress}_d(x) = a\}, \quad (32)$$

Using a correlated random mask r , the parties can open $\delta = x - r \pmod{q}$, and check $x \in I$ by equivalently checking $r \in I - \delta$, which does not reveal information about x because r is a fresh uniformly random value.

This step reduces message decoding from 28 to 6 rounds, and compressed equality from 18 to 6 rounds. The overall round complexity of threshold decapsulation is reduced to 360 from the baseline 384.

4.2.2 Parallel PRF Expansion. Our next observation is that, once the reencryption randomness r' is known, the five SHAKE256 PRF evaluations used to derive the secret and error polynomials are independent and can be evaluated in parallel. Specifically, the two calls with $\eta_1 = 3$ each needs two Keccak-f[1600] permutations, while the other three using $\eta_2 = 2$ needs one permutation each.

Although seven Keccak permutations are evaluated in total, their dependency depth is only 2, since independent permutations can be evaluated in parallel

$$R_{\text{PRF}}^{\text{sequential}} = 7 \cdot 24 = 168, \quad R_{\text{PRF}}^{\text{parallel}} = 2 \cdot 24 = 48. \quad (33)$$

This step reduces the number of rounds from 360 to 240.

4.2.3 Global Scheduling of the FO branches. Our third observation is that the fallback value $J(z||c)$ depends only on the secret z and the public ciphertext. Therefore, it can start at the beginning rather than waiting for message recovery and reencryption. Given the preceding optimisations, the ciphertext validation branch now has depth 95 whereas the fallback one is much longer of 144.

Running the two branches concurrently and joining only at the final secret selection step yields the reduced round complexity of

$$R_A^{\text{dec}} = \max\{95, 144\} + 1 = 145. \quad (34)$$

We note that even using unoptimised 18 round decoding and 28 round equality circuits, the ciphertext validation branch has depth only 119 rounds, which is still shorter than the fallback branch. Nevertheless, the ML-KEM specific predicate optimisations are still useful in implementation, as they reduce the overall computation and communication in the ciphertext validation branch.

4.2.4 Full ML-KEM Compatibility. These optimisations only change the schedule of ML-KEM decapsulation algorithm, while the standard encapsulation remains unchanged. The resulting TKEM scheme A-512 has the same public key and ciphertext format as ML-KEM-512. Thus, its public key and ciphertext sizes are 800 bytes and 768 bytes, respectively. The cost of compatibility is due to the 144-round secret fallback branch.

4.3 Optimisation in Scheme B

Scheme A reaches the limit of round optimisation for the MPC approach to thresholdise ML-KEM decapsulation. To improve round complexity further, we need to change the MPC heavy operations in ML-KEM. Our scheme B will use more MPC friendly variant of Kyber PKE and hash functions. Additional decapsulation circuit scheduling optimisations will allow us to reduce the round complexity to 35.

4.3.1 MPC friendly PKE. Scheme B uses an uncompressed module lattice variant of the LPR encryption scheme [22], instantiated by the same lattice and noise parameters with ML-KEM-512. We use canonical uncompressed ring coefficients. Its public key contains an explicit 2×2 matrix $\mathbf{A}$ together with a vector $\mathbf{t}$. Its ciphertext $(\mathbf{u}, v)$ contains three canonically encoded ring polynomials.

This removes ciphertext compression from the ciphertext validation path. Equality of a secret coefficient with the corresponding public coefficient can be tested using a masked opening, which produces one equality bit for each of the 768 ciphertext coefficients. A depth 4 reduction circuit then combines these 768 equality bits into a single ciphertext validity bit. This gives $R_B^{\text{Eq}} = 15$.

4.3.2 MPC friendly Hash and Scheduling. We instantiate all hashing operations on secret shared inputs with KT128, the 128-bit security Kangaroo Twelve function specified in RFC 9861 [31]. Distinct customisation strings provide domain separation between its different uses.

Five independent KT128 expansion calls jointly derive the 32-byte pre-key $\bar{K}$ and the 6144 pseudorandom bits required for centered binomial distribution (CBD) sampling. Each call has a short input and produces at most 168 bytes, which requires only one Keccak-p[1600, 12] permutation. Since the five calls are independent, they can be evaluated in parallel. This gives $R_B^{\text{Expand}} = 12$.

The CBD sampling bits and the recovered message bits required by $\text{EncodeMsg}(m')$ are converted from Boolean to arithmetic sharing, together in a single batched opening round. After the expansion, two independent computations proceed in parallel. The good key branch derives the candidate shared key from the pre-key, while

the validation branch performs CBD sampling, reencryption, and ciphertext equality testing.

The validation branch costs $R_B^{\text{valid}} = 16$ rounds, while the domain separated good and bad key derivations execute in parallel in $R_B^{\text{KDF}} = 12$ rounds. With a final secret mux, the MPC protocol for threshold decapsulation requires $R_B^{\text{dec}} = 6 + 12 + \max\{16, 12\} + 1 = 35$ rounds.

5 MPC Building Blocks

In this section, we describe threshold evaluation protocol for each operation in ML-KEM-512 decapsulation. We will specify the data representation and communication of each operation in a modular fashion so that scheme A will be able assemble these primitives and exploit concurrency between independent calls to build a complete threshold decapsulation protocol. Our MPC protocols use Shamir secret sharing with Beaver multiplication, correlated randomness provided by a preprocessing phase which is done by a trusted party.

5.1 Sharing Domains and Communication Cost

5.1.1 Arithmetic Sharing. Let $S \subseteq \{1, \dots, N\}$ be the active set. We write $s = |S| \geq t$. For $x \in \mathbb{F}_q$, a Shamir secret sharing $[x]_q$ is obtained from a uniformly random polynomial $p_x \in \mathbb{F}_q[X]$ of degree at most $t - 1$ satisfying $p_x(0) = x$ [30]. Party P_i holds the value $p_x(i)$. Any active set of at least t parties can reconstruct $x = \sum_{i \in S} \lambda_i^S p_x(i)$, where λ_i^S is the public Lagrange coefficient for interpolation at zero. Additions, subtractions, multiplication by a public scalar, NTT, inverse NTT, and every other public linear map are performed locally on the shares.

5.1.2 Boolean Sharing. Boolean values are embedded as 0 and 1 in the field $\mathbb{F}_{2^\kappa}$. We choose $\kappa = \min\{k : 2^k - 1 \geq N\}$ so that the field contains at least N distinct nonzero evaluation points. $[b]_2$ denotes a Shamir sharing over $\mathbb{F}_{2^\kappa}$ of the secret bit $b \in \{0, 1\}$. XOR is field addition and local. AND of two secret bits is field multiplication which requires interactions. The Kyber PKE arithmetic are over $\mathbb{F}_q$, while SHA3/SHAKE and comparison circuits are binary. We will need conversion protocols to move between these settings.

5.1.3 Cost Model. For an MPC protocol Π , we denote the number of its rounds by $R(\Pi)$. We write $M_2(\Pi)$, $O_q(\Pi)$, and $O_2(\Pi)$ for the number of Beaver multiplications in $\mathbb{F}_{2^\kappa}$, directly opened $\mathbb{F}_q$ elements, and directly opened $\mathbb{F}_{2^\kappa}$ elements outside Beaver multiplication, respectively. The two masked values opened by each Boolean Beaver multiplication are counted in M_2 , not O_2 .

Using packed communication, a batch containing M_2 Boolean multiplications, O_2 additional Boolean openings, and O_q arithmetic openings contributes $C = s(2M_2\kappa + O_2\kappa + 12O_q)$ bits to aggregate online communication.

5.2 Beaver Multiplication

All nonlinear Boolean gates are evaluated using Beaver multiplication [2]. Let $\mathbb{F}$ be either $\mathbb{F}_q$ or $\mathbb{F}_{2^\kappa}$. The dealer supplies fresh sharings $[a]_{\mathbb{F}}$, $[b]_{\mathbb{F}}$, $[c]_{\mathbb{F}}$ where $c = ab$ over $\mathbb{F}$. To multiply shared x and y , the parties open $d = x - a$, $e = y - b$, then compute locally $[xy]_{\mathbb{F}} = [c]_{\mathbb{F}} + d[b]_{\mathbb{F}} + e[a]_{\mathbb{F}} + de$. The two openings can be batched into one interaction round. For a batch of L Boolean multiplications, the parties need L Boolean triples, one communication round, and

communicate $2Ls$ elements of $\mathbb{F}_{2^\kappa}$. This lane-wise batching is the unit for counting Boolean circuit depth below.

5.3 Conversion and Predicate Evaluation

5.3.1 B2A Conversion. Suppose that we have a bit $b \in \{0, 1\}$ with Boolean shares $[b]_2$, but subsequently need to work with it over $\mathbb{F}_q$. The dealer samples a uniform bit r and supplies correlated sharings $[r]_2$ and $[r]_q$. The parties open $d = b \oplus r$. Since d is a public bit, the parties can set

$$[b]_q = \begin{cases} [r]_q, & d = 0, \\ 1 - [r]_q, & d = 1. \end{cases} \quad (35)$$

Because r is uniformly random, the opened d is uniformly random, and independent of b . A batch of L conversions costs one round and Ls opened Boolean field elements $R_{B2A}(L) = 1$, $M_2 = 0$, $O_2 = L$.

5.3.2 Masked Predicates. Let x be a secret coefficient in $\mathbb{Z}_q$. The dealer samples $r \in \mathbb{Z}_q$ and supplies $[r]_q$ together with Boolean sharings of its 12 canonical bits. The parties open $\delta = x - r \pmod{q}$. For any public set $I \subseteq \mathbb{Z}_q$, we define

$$I - \delta := \{y - \delta \pmod{q} : y \in I\}. \quad (36)$$

Then, $x \in I$ if and only if $r \in I - \delta$. We now need to evaluate a Boolean predicate on the preprocessed bits of the random mask r and public endpoints determined by δ .

For compression predicates, we define

$$I_{a,d} := \{x \in \mathbb{Z}_q : \text{Compress}_d(x) = a\}. \quad (37)$$

For ML-KEM, $I_{a,d}$ is a cyclic interval in $\mathbb{Z}_q$. Consequently, we have $\text{Compress}_d(x) = a$ if and only if $r \in I_{a,d} - \delta$.

5.3.3 Comparison. We use one common Boolean circuit for all interval predicates. Let $r = (r_{11} \dots r_{10})_2$ be a 12-bit secret and c be a public integer in $[0, 3329]$. For each bit position, we define a state (e_i, g_i) , where e_i saying that the one-bit prefixes are equal and g_i saying the secret bit is greater than the public one. Because the public bit c_i is known, the computations $e_i = \neg(r_i \oplus c_i)$ and $g_i = r_i \wedge \neg c_i$ can be done locally.

If (e_H, g_H) summarises a more significant segment and (e_L, g_L) summarises the remaining segment, the concatenated state is $e = e_H e_L$, $g = g_H \oplus e_H g_L$ which is valid, because $g_H = 1$ implies $e_H = 0$, so that both terms cannot be 1. Each merge uses two secret ANDs. A balanced tree on 12 one-bit states has 11 merges and depth $\lceil \log_2 12 \rceil = 4$. Therefore one comparison $r \geq c$ costs $M_2(\text{GE}_{12}) = 22$, $R(\text{GE}_{12}) = 4$.

To test a public cyclic interval represented by endpoints (L, U) , we evaluate the two comparisons $b_L = 1[r \geq L]$, $b_U = 1[r \geq U + 1]$ in parallel. For a non-wrapping interval $L \leq U$, membership is $b_L \wedge \neg b_U$. For a wrapping interval $L > U$, membership is $b_L \vee \neg b_U$. The wrap condition is public, so the same circuit can use one final secret AND and public XORs to select the appropriate expression. After the masked arithmetic opening, one interval membership test has $R = 6$, $M_2 = 45$, $O_q = 1$. We note that arbitrarily many independent interval tests can be packed lanewise and require only depth 6.

MPC Protocols for Operations in ML-KEM Decapsulation. Given these MPC building blocks, we construct MPC protocol for each operation in the decapsulation algorithm of ML-KEM-512.

Table 2: ML-KEM operations and their threshold protocols. R, M_2, O_q, O_2 are the numbers of rounds, Beaver triples, openings over $\mathbb{F}_q$, and openings over $\mathbb{F}_{2^\kappa}$, respectively.

Operation	Protocol	R	M_2	O_q	O_2
ExpandA, public parsing	Π_{ExpandA}	0	0	0	0
DecLin	Π_{DecLin}	0	0	0	0
DecodeMsg	Π_{Decode}	6	11520	256	0
G	Π_G	24	38400	0	0
J	Π_J	144	230400	0	0
ExpandCoins	Π_{Coins}	49	268800	6144	0
EncodeMsg	Π_{Encode}	1	0	0	256
ReEncLin	Π_{ReEnc}	0	0	0	0
CompEq	Π_{CompEq}	16	35327	768	0
Select	Π_{Mux}	1	256	0	0

We summarise the protocols and their cost accounting in Table 2. Detailed protocol description is given in Appendix A.3.

6 Scheme A: ML-KEM Compatible TKEM

In this section, we combine MPC threshold primitives from Section 5 into scheme A to obtain a TKEM.

6.1 Construction

6.1.1 Threshold Key. Let $dk = dk_{\text{PKE}} \| ek \| h \| z$ be a validated ML-KEM-512 decapsulation key, where $dk_{\text{PKE}} = \text{ByteEncode}_{12}(\widehat{s})$, $h = H(ek)$, and $z \in \mathbb{B}^{32}$. In a setup phase, a trusted dealer parses the key and distributes coefficientwise arithmetic Shamir secret shares $[\widehat{s}[i]_j]_q$, $i \in \{0, 1\}$, $j \in \{0, \dots, 255\}$, and Boolean domain shares of the 256 bits of z as $[z_0]_2, \dots, [z_{255}]_2$. The ek and h remain public. We denote the resulting threshold key by

$$\text{tdk}_A = ([\widehat{s}]_q, [z]_2; ek, h). \quad (38)$$

6.1.2 Masked Predicates. For message decoding and ciphertext validation check, we use masked predicate method from Subsection 5.3.2. This helps reduce the size of Boolean representation of message decoding, and compressed candidate ciphertext for validation check. This improves threshold decapsulation by 24 rounds. It also replaces generic conversions by 256 masked arithmetic openings for decoding and 768 masked arithmetic openings for compressed equality check. Those work and communication savings are useful for the whole decapsulation protocol.

6.1.3 Parallel Pseudorandom Expansion. Once r' has been produced by $G(m' \| h)$, the five calls defining $\text{ExpandCoins}(r')$ have no data dependencies on one another. We schedule them to run in parallel. Their *work* remains seven Keccak permutations, but their *depth* is the maximum depth of a single call. The 6144 Boolean CBD bits produced by these PRFs and the 256 decoded message bits are then converted together in a single 6400-bit B2A batch. This optimisation reduces round complexity by 120 rounds.

6.1.4 Scheduling the FO Branch. The fallback key computation $K_\perp = J(z \| c)$ depends only on the secret z and public ciphertext, and is independent of other operation. We now schedule both the ciphertext validation and fallback branches to start at the same time at beginning of decapsulation. The total number of round is

Algorithm 2 Scheme A for ML-KEM-512.

Require: $\text{tdk}_A = ([\widehat{s}]_q, [z]_2; ek, h), c, \mathcal{S}$
Ensure: Secret shared 256-bit session key $[K]_2$
1: Parse and decompress c publicly
2: $[w]_q \leftarrow \Pi_{\text{DecLin}}([\widehat{s}]_q, c)$
3: **Run two branches in parallel:**
 Ciphertext validation branch:
 $[m']_2 \leftarrow \Pi_{\text{Decode}}([w]_q)$
 $([K']_2, [r']_2) \leftarrow \Pi_G([m']_2, h)$
 $[\omega]_2 \leftarrow \Pi_{\text{Coins}}^{\text{PRF}}([r']_2)$
 Convert $([\omega]_2, [m']_2)$ in one 6400-bit B2A batch
 Locally derive $[y']_q, [e'_1]_q, [e'_2]_q$ by CBD
 Locally derive $[\mu']_q = [\text{EncodeMsg}(m')]_q$
 $([u']_q, [v']_q) \leftarrow \Pi_{\text{ReEnc}}(ek, [y']_q, [e'_1]_q, [e'_2]_q, [\mu']_q)$
 $[b]_2 \leftarrow \Pi_{\text{CompEq}}([u']_q, [v']_q, c)$
 Fallback branch:
 $[K_\perp]_2 \leftarrow \Pi_J([z]_2, c)$
4: $[K]_2 \leftarrow \Pi_{\text{Mux}}([b]_2, [K']_2, [K_\perp]_2)$
5: **return** $[K]_2$

now $R_A^{\text{dec}} = 145$. Reconstructing the 256-bit session key needs one additional batched opening round $R_A^{\text{out}} = 146$.

We present the final threshold decapsulation protocol scheme A in Algorithm 2, where $\Pi_{\text{Coins}}^{\text{PRF}}$ denotes the five threshold SHAKE256 calls up to their 6144 raw Boolean CBD bits, before the standalone B2A step of Π_{Coins} .

6.2 Analysis

6.2.1 Correctness. Assume that the preprocessing phase is correct and the active set $\mathcal{S}$ has $|\mathcal{S}| \geq t$. Any reconstructed session key of scheme A's output equals the decapsulated key of ML-KEM-512 on the same validated ciphertext. This can be verified by checking output of each operation in scheme A one by one. We provide detailed analysis in Appendix B.2.

6.2.2 Compatibility. Since reconstruction of the threshold protocol for each operation in scheme A (Algorithm 2) produces exactly the corresponding decapsulation operation output in ML-KEM-512 (Algorithm 1), the final reconstruction returns the correct session key, including implicit rejection case. Therefore, a standard ML-KEM encapsulator can communicate with scheme A without any change to the encapsulation key and ciphertext format, encapsulation algorithm, or output format.

6.2.3 Complexity. Scheme A requires $R_A^{\text{dec}} = 145$ rounds of interaction. Our scheduling scheme changes the depth of decapsulation circuit, but keep the operation count the same. Therefore, the number of online Boolean multiplications is $M_{2,A} = 584703$. It opens $O_q = 1024$ ($O_2 = 6400$) masked (conversion) values for each active party. Since each Boolean multiplication opens two masked values, the total number of Boolean field contributions before key reconstruction is $2M_{2,A} + O_2 = 1175806$. Using κ packed bits for each contribution and 12 bits for each $\mathbb{F}_q$ contribution, the aggregate packed online communication cost is $C_A^{\text{dec}} = s(1175806\kappa + 12 \cdot 1024)$ bits. The key reconstruction step requires one addition step of batched reconstruction of the 256 Boolean key bits, which adds $256\kappa s$ bits of communication.

7 Scheme B: An Efficient MPC-Friendly TKEM

In this section, we present a TKEM scheme B which explores MPC friendly options in the design space of Kyber [5] to improve round

complexity. First, we will change the PKE by removing compression and interval predicate in Kyber, making ciphertext validation into exact equality checks. This gives a module version of the LPR encryption scheme [22]. Second, we will use a more depth efficient hash function KT128 as specified by RFC 9861 [31].

7.1 Uncompressed Kyber PKE and a KEM

Scheme B works over the same ring as ML-KEM-512 $n = 256$, $q = 3329$, $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$, and also uses parameters $k = 2$, $\eta_1 = 3$, $\eta_2 = 2$. χ_η denotes the coefficient-wise distribution CBD_η on R_q . Message encoding represents a message bit 0/1 by 0/1665. Scheme B stores the matrix $\mathbf{A} \xleftarrow{\$} R_q^{2 \times 2}$ explicitly, not by seed expansion.

7.1.1 Canonical 12-bit Encoding. Coefficients of polynomial in R_q are represented by unique integers in $\{0, \dots, q-1\}$, encoded in exactly 12 bits. For a polynomial $a(X) = \sum_{i=0}^{255} a_i X^i \in R_q$, $a_i \in \{0, \dots, q-1\}$, we define $\text{EncPoly}(a) \in \mathbb{B}^{384}$ by packing each pair a_{2j}, a_{2j+1} into three bytes $b_{3j}, b_{3j+1}, b_{3j+2}$ as

$$b_{3j} = a_{2j} \pmod{2^8}, \quad (39)$$

$$b_{3j+1} = \left\lfloor \frac{a_{2j}}{2^8} \right\rfloor + 2^4(a_{2j+1} \pmod{2^4}), \quad (40)$$

$$b_{3j+2} = \left\lfloor \frac{a_{2j+1}}{2^4} \right\rfloor, \quad 0 \leq j < 128. \quad (41)$$

Conversely, DecPoly first recovers

$$\tilde{a}_{2j} = b_{3j} + 2^8(b_{3j+1} \pmod{2^4}), \quad (42)$$

$$\tilde{a}_{2j+1} = \left\lfloor \frac{b_{3j+1}}{2^4} \right\rfloor + 2^4 b_{3j+2}, \quad (43)$$

and accepts if and only if every recovered value satisfies $\tilde{a}_i < q$. Note that values in $\{q, \dots, 4095\}$ are rejected rather than reduced modulo q . Thus, EncPoly is an injective encoding.

For $ek = (\mathbf{A}, \mathbf{t})$ with $\mathbf{A} = (A_{ij})_{i,j \in \{0,1\}}$ and $\mathbf{t} = (t_0, t_1)$, we define the public key encoding as

$$\begin{aligned} \text{EncPK}(ek) = & \text{EncPoly}(A_{00}) \parallel \text{EncPoly}(A_{01}) \parallel \text{EncPoly}(A_{10}) \\ & \parallel \text{EncPoly}(A_{11}) \parallel \text{EncPoly}(t_0) \parallel \text{EncPoly}(t_1). \end{aligned} \quad (44)$$

which gives public key size $|ek| = 2304$ bytes. For a ciphertext $c = (\mathbf{u}, v)$ where $\mathbf{u} = (u_0, u_1)$, we use the encoding

$$\text{EncCT}(c) = \text{EncPoly}(u_0) \parallel \text{EncPoly}(u_1) \parallel \text{EncPoly}(v), \quad (45)$$

which gives $|c| = 1152$ bytes.

For API checks before decapsulation, public key and ciphertext parsing will check lengths, then applies canonical decoder to each polynomial. For validated objects, equality can be checked by coefficient-wise equality. Decapsulation key has the form $dk = (ek, s, z)$, where $s \in R_q^2$ is the secret, and $z \in \mathbb{B}^{32}$ is the implicit rejection secret. We use the secret encoding

$$\text{EncPriv}(s, z) = \text{EncPoly}(s_0) \parallel \text{EncPoly}(s_1) \parallel z, \quad (46)$$

where z is written as 32 bytes. The whole secret input has size $|\text{EncPriv}(s, z)| = 800$ bytes.

7.1.2 KT128 Functions. Scheme B uses the KT128 hash with the customisation string interface, as specified in RFC 9861 in [31]. For message M , customisation string C and requested output length L bytes, the call $\text{KT128}(M, C, L)$ first forms the reversible input $S = M \parallel C \parallel \text{length_encode}(|C|)$. If $|S| \leq 8192$, KT128 is the single

node call $\text{TurboSHAKE128}(S, 07, L)$, where TurboSHAKE128 [31] uses Keccak- $p[1600, 12]$ and a 168 byte rate. Longer public inputs are handled by a tree mode which is public computation.

We use the following five customisation strings

$$C_{ek} = \text{SchemeB-K12:H_ek}, \quad C_c = \text{SchemeB-K12:H_c}, \quad (47)$$

$$C_G = \text{SchemeB-K12:G}, \quad (48)$$

$$C_g = \text{SchemeB-K12:K_good}, \quad C_b = \text{SchemeB-K12:K_bad}. \quad (49)$$

which are interpreted as their ASCII byte strings. The public digests are

$$H_{ek}(ek) := \text{KT128}(\text{EncPK}(ek), C_{ek}, 32), \quad (50)$$

$$H_c(c) := \text{KT128}(\text{EncCT}(c), C_c, 32). \quad (51)$$

7.1.3 Message and Public Digest Expansion. Given a 32 byte message m and the public digest $h = H_{ek}(ek)$, we define five domain separated chunks

$$Y_j := \text{KT128}(m \parallel h \parallel \langle j \rangle_8, C_G, L_j), \quad 0 \leq j \leq 4, \quad (52)$$

where the chunk lengths are $L_j = 168$ for $j = 0, 1, 2, 3$, and $L_4 = 128$. Here, the lengths are chosen for both MPC scheduling and CBD sampling. For each chunk, the RFC 9861 single node string has at most 168 bytes. Since $L_j \leq 168$, each chunk is exactly one Keccak- $p[1600, 12]$ permutation, and all five chunks have identical interaction depth.

The index $\langle j \rangle_8$ is exactly one byte. The customisation string C_G has 13 bytes and $\text{length_encode}(13)$ has 2 bytes, so that each chunk has the same single node input length 80 bytes. Since $L_j \leq 168$ for every j , each secret expansion call fits in one TurboSHAKE128 rate block both on input and requested output. We then concatenate $Y = Y_0 \parallel Y_1 \parallel Y_2 \parallel Y_3 \parallel Y_4$ with $|Y| = 800$ bytes, and parse $Y = \bar{K} \parallel \omega$, where $|\bar{K}| = 32$, $|\omega| = 768$.

The string ω is parsed into 5 parts which used sequentially as input for centered binomial distribution sampling

$$\omega = \omega_{y_0} \parallel \omega_{y_1} \parallel \omega_{e_{1,0}} \parallel \omega_{e_{1,1}} \parallel \omega_{e_2}, \quad (53)$$

with $|\omega_{y_0}| = |\omega_{y_1}| = 192$ bytes and $|\omega_{e_{1,0}}| = |\omega_{e_{1,1}}| = |\omega_{e_2}| = 128$. The resulting sampled polynomials are

$$y_i = \text{SamplePolyCBD}_3(\omega_{y_i}), \quad i \in \{0, 1\}, \quad (54)$$

$$e_{1,i} = \text{SamplePolyCBD}_2(\omega_{e_{1,i}}), \quad i \in \{0, 1\}, \quad (55)$$

$$e_2 = \text{SamplePolyCBD}_2(\omega_{e_2}). \quad (56)$$

We denote the expansion procedure as

$$\text{BExpand}(m, h) := (\bar{K}, \mathbf{y} = (y_1, y_2), \mathbf{e}_1 = (e_{1,0}, e_{1,1}), e_2). \quad (57)$$

7.1.4 Scheme B's PKE. We use the following uncompressed variant of Kyber as the based PKE. Key generation samples $\mathbf{A} \xleftarrow{\$} R_q^{2 \times 2}$, $\mathbf{s}, \mathbf{e} \xleftarrow{\$} \chi_3^2$, then forms $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$. The public key is $ek = (\mathbf{A}, \mathbf{t})$. To encrypt a message m using expanded coins $(\mathbf{y}, \mathbf{e}_1, e_2)$ as above, we compute

$$\mathbf{u} = \mathbf{A}^T \mathbf{y} + \mathbf{e}_1, \quad v = \mathbf{t}^T \mathbf{y} + e_2 + \mu(m), \quad (58)$$

and output the ciphertext $c = (\mathbf{u}, v)$. We denote this deterministic encryption map by

$$\text{BEnc}(ek, m, \mathbf{y}, \mathbf{e}_1, e_2) := (\mathbf{u}, v). \quad (59)$$

Algorithm 3 Threshold decapsulation in Scheme B.

Require: $ek = (A, t), c = (u, v), [s]_q, [z]_2$
Ensure: Boolean sharing $[K]_2$
1: Locally compute $h \leftarrow H_{ek}(ek)$ and $d \leftarrow H_c(c)$
2: Locally compute $[w]_q \leftarrow v - [s]_q^T u$
3: $[m']_2 \leftarrow \Pi_{\text{Decode}}([w]_q)$
4: $([\bar{K}']_2, [\omega']_2) \leftarrow \Pi_{\text{BExpand}}([m']_2, h)$
Ciphertext validation branch:
5: Batched opening $[y']_q, [e'_1]_q, [e'_2]_q, [\mu(m')]_q$
6: Locally derive $[y']_q, [e'_1]_q, [e'_2]_q, [\mu(m')]_q$
7: Locally compute $[c']_q = ([u']_q, [v']_q)$
8: $[b]_2 \leftarrow \Pi_{\text{BEq}}([c']_q, c)$
Key derivation branch, in parallel:
9: $([K_{\text{good}}]_2, [K_{\text{bad}}]_2) \leftarrow \Pi_{\text{BKDF}}([\bar{K}']_2, [z]_2, d)$
10: $[K]_2 \leftarrow \Pi_{\text{Mux}}([b]_2, [K_{\text{good}}]_2, [K_{\text{bad}}]_2)$
11: **return** $[K]_2$

Given $dk = (ek, s, z)$, decryption computes

$$\text{BDecLin}(s, c) := v - s^T u, \quad (60)$$

then recover the message

$$m' = \text{DecodeMsg}(\text{BDecLin}(s, c)), \quad (61)$$

using the interval $833 \leq x \leq 2496$ for decoding.

For an honestly generated ciphertext, one has

$$\text{BDecLin}(s, c) = \mu(m) + v, \quad v = e^T y + e_2 - s^T e_1, \quad (62)$$

which does not contain ciphertext-compression error. The condition for correct message recovery is $|v_i| < 832$, where $|\cdot|$ denotes centered representation modulo q .

7.1.5 Scheme B's KEM. We now obtain a KEM scheme by applying the FO transform with implicit rejection. The two secret key derivation calls are

$$K_{\text{good}} := \text{KT128}(\bar{K} \| d, C_g, 32), \quad K_{\text{bad}} := \text{KT128}(z \| d, C_b, 32), \quad (63)$$

where $d = H_c(c)$. The final session key is selected based on ciphertext validation check. We present this KEM scheme in Algorithm 7.

7.2 Construction of Scheme B

We apply Shamir secret sharing and MPC techniques to thresholdise the decapsulation algorithm of this KEM. The construction combines the MPC building blocks to construct a threshold protocol for each operation in scheme B's KEM decapsulation. We summarise the protocols and their MPC cost in Table 5. Detailed constructions are given in Appendix A.5. The full scheme B is presented in Algorithm 3.

8 Security

Our security argument for the TKEM schemes A and B follows by composition of security of the MPC layer and the underlying KEMs. In the static semi-honest model, since Shamir secret sharing and Beaver multiplication perfectly hide all intermediate values, the MPC layer does not reveal anything beyond the prescribed KEM decapsulation. This provides information theoretic security for the MPC layer. Thus, Scheme A can inherit ML-KEM security by exact composition. For Scheme B, we change the underlying PKE and KEM, which requires additional security analysis. In this section, we summarise the security arguments, while complete security proofs are deferred to Appendix B.

8.1 Security Model

We first formalise the threshold IND-CCA security for a setup assisted threshold decapsulation protocol of for a given KEM. It modularly combines CCA security of the KEM with the internal view of static passive coalition of threshold parties. This is specific to our construction, and covers narrower security scope than general threshold PKE or TKEM security notions with malicious parties or adaptive corruptions [9].

8.1.1 Roles of the Participants.

- **Master private key owner** generates public-private key pair and owns the private key. He distributes the shares of the private key to the threshold parties securely before the TKEM process.
- **Trusted MPC setup $\mathcal{D}$** is a trusted honest offline party. During setup, it generates the correlated randomness, including Beaver triples and correlated masks, for MPC protocols. This one-time preprocessing step is independent of public key and ciphertext. After the offline setup step, $\mathcal{D}$ does not take part in the online decapsulation protocol. It also does not receive the ciphertext or reconstructed session key.
- **Threshold decapsulation parties $P_1, \dots, P_N$.** P_i receives a decapsulation key share tdk_i and preprocessing string from the setup. An active set of parties with indices in $S \subseteq [N]$, $|S| \geq t$ can jointly execute the interactive threshold decapsulation protocol.
- **Encapsulator** is any external sender who can run the ordinary encapsulation algorithm $\text{Encaps}(ek)$ (ML-KEM encapsulation in our scheme A). It does not interact with the setup or the parties.
- **Caller** is any external party who participates in the KEM process to send a public ciphertext c or receive the reconstructed session key produced by the parties, but it does not see internal state of the threshold protocol. In the chosen ciphertext security experiment, the adversary is a caller who submits decapsulation query

8.1.2 Setup assisted TKEM syntax.

$$\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps}) \quad (64)$$

with session key space $\mathcal{K}$, we define a TKEM based on KEM as

$$\text{TKEM} = (\text{KeyGen}, \text{Encaps}, \text{DShare}, \text{DPrep}, \Pi_{\text{TDec}}, \text{Open}), \quad (65)$$

where

- $\text{DShare}_{N,t}(dk)$ is executed only by the setup $\mathcal{D}$ to produce the key shares $(tdk_1, \dots, tdk_N)$. Then, $\mathcal{D}$ privately sends tdk_i to P_i .
- $\text{DPrep}_{N,t}(1^\lambda)$ is also executed by $\mathcal{D}$ to produce one preprocessing string $\rho = (\rho_1, \dots, \rho_N)$ for each run of the online MPC protocol. ρ is independent of (ek, dk) and any ciphertext.
- Π_{TDec} can be run by any active set of parties $S \subseteq [N]$, $|S| \geq t$. On public input (ek, c) and private input $(tdk_i, \rho_i)_{i \in S}$, the parties jointly evaluate Decaps. Each parties obtain an output shares $(out_i)_{i \in S}$ of the session key.
- $\text{Open}_S((out_i)_{i \in S})$ is a key reconstruction procedure which returns the whole session key K .

The setup is an abstraction for the procedure of key generation and sharing in the setup phase. In real applications, it can be a trusted manager, or replaced by distributed protocols for key generation and sharing. We write

$$(ek, dk) \leftarrow \text{KeyGen}(1^\lambda), \quad (66)$$

$$(tdk_1, \dots, tdk_N) \leftarrow \text{DShare}_{N,t}(dk). \quad (67)$$

8.1.3 API format checks. ML-KEM's API requires public parsing and format checks before any decapsulation. Let $\text{ValidCT}(ek, c)$ be a deterministic checks depending only on public encoding of ek , c . If $\text{ValidCT}(ek, c) = 0$, the system returns a fixed public error symbol $\perp_{\text{pub}}$ without invoking Π_{TDec} . If $\text{ValidCT}(ek, c) = 1$, one can run the threshold protocol. For Scheme B, public canonical decoding is also part of ValidCT .

8.1.4 Correctness. The threshold protocol is an exact realisation of Decaps if for every key pair (ek, dk) output by KeyGen , valid ciphertext c , active set $S \subseteq [N]$, $|S| \geq t$, sharing output by $\text{DShare}_{N,t}$, and honestly generated preprocessing string, it holds that

$$\text{Open}_S(\Pi_{\text{TDec}}((tdk_i, \rho_i)_{i \in S}; ek, c)) = \text{Decaps}(dk, c). \quad (68)$$

If Decaps has correctness error at most $\delta(\lambda)$, then we also require the threshold protocol to open correctly with probability at least $1 - \delta(\lambda)$.

8.1.5 Static semi-honest model. We consider an adversary who can corrupt a fixed set $C \subseteq [N]$, $|C| < t$ which is chosen after ek is known but before any setup output is revealed. C cannot be changed during the experiment. Corrupted parties follow Π_{TDec} , but reveal their internal state to the adversary. For an online decapsulation protocol execution indexed by v with respect to the active set S and ciphertext c , the corrupted party view contains

$$\text{View}_C^v(S, c) := ((tdk_i)_{i \in C}, (\rho_i^v)_{i \in C}, (\omega_i^v)_{i \in C \cap S}, \text{Msg}_{C \cap S}^v, \text{OpenVals}^v, (\text{out}_i^v)_{i \in C \cap S}, \text{RecTr}^v), \quad (69)$$

where ω_i^v is the local online randomness of P_i which is empty if the required randomness is supplied by preprocessing. $\text{Msg}_{C \cap S}^v$ contains all messages sent or received by corrupted active parties, OpenVals^v contains every publicly reconstructed masked value, and RecTr^v contains any transcript of the final key reconstruction.

We assume that corrupted parties are semi-honest so that the adversary cannot tell them to send inconsistent shares, deviate from protocol.

8.1.6 Offline preprocessing. is independent of ek and c . In the security experiment, this is captured by giving a preprocessing oracle $\mathcal{O}_{\text{Prep}}$ which on receiving a query will ask $\mathcal{D}$ to sample a fresh string

$$\rho^v = (\rho_1^v, \dots, \rho_N^v) \leftarrow \text{DPrep}_{N,t}(1^\lambda), \quad (70)$$

stores it with index v , then returns $(v, (\rho_i^v)_{i \in C})$ to the adversary. The adversary can inspect all corrupted preprocessing shares for choosing ciphertext.

8.1.7 Threshold decapsulation oracle. $\mathcal{O}_{\text{TDec}}(v, S, c)$ checks if the index v is not previously queried and $S \subseteq [N]$ with $|S| \geq t$. If the check fails, it returns $\perp_{\text{pub}}$, leaves v unused. Otherwise it executes Π_{TDec} using the stored ρ^v , reconstructs $K = \text{Open}_S((\text{out}_i^v)_{i \in S})$, marks v as queried, then sends to the adversary both the decapsulation key K and the corrupted party view $\text{View}_C^v(S, c)$, which

models both the chosen ciphertext oracle and additional information created by distributed decapsulation.

8.1.8 TIND-CCA security. The TIND-CCA experiment with adversary $\mathcal{A}$ is as follows.

- (1) The challenger generates $(ek, dk) \leftarrow \text{KeyGen}(1^\lambda)$ and gives ek to $\mathcal{A}$.
- (2) The adversary chooses a corruption set $C \subseteq [N]$ satisfying $|C| < t$. The set C is fixed for the rest of the experiment.
- (3) The trusted setup computes $(tdk_1, \dots, tdk_N) \leftarrow \text{DShare}_{N,t}(dk)$, and the challenger gives $(tdk_i)_{i \in C}$ to $\mathcal{A}$. The setup's remaining internal state and honest party's secret state remain hidden.
- (4) The adversary obtains adaptive access to $\mathcal{O}_{\text{Prep}}$ and $\mathcal{O}_{\text{TDec}}$.
- (5) At challenge time, the challenger samples a uniform bit $b \leftarrow \{0, 1\}$, runs ordinary encapsulation $(K^*, c^*) \leftarrow \text{Encaps}(ek)$, and independently samples $U \leftarrow \mathcal{K}$. It sets

$$\tilde{K}^* = \begin{cases} K^*, & b = 0, \\ U, & b = 1, \end{cases} \quad (71)$$

then gives $(c^*, \tilde{K}^*)$ to $\mathcal{A}$.

- (6) The adversary continues to have adaptive access to both oracles so that no post-challenge query to $\mathcal{O}_{\text{TDec}}$ may contain the queried $c = c^*$.
- (7) Finally, $\mathcal{A}$ outputs a guessing bit b' . The experiment outputs 1 if and only if $b' = b$.

The advantage of $\mathcal{A}$ is defined as

$$\text{Adv}^{\text{TIND-CCA}}(\mathcal{A}) := |\Pr[b' = b] - \frac{1}{2}|. \quad (72)$$

Definition 1. A setup assisted TKEM is TIND-CCA secure in the static semi-honest model if it satisfies the correctness in Eq. (68) and for every PPT adversary $\mathcal{A}$, admissible (N, t) , and static corruption set chosen as above, the advantage of $\mathcal{A}$ in Eq. (72) is negligible.

8.2 Privacy of MPC Protocols

For our TKEM schemes, every online interaction used in our protocols is perfectly masked. Specifically, fewer than t Shamir shares have a secret independent marginal. Beaver openings reveal only $x - a$ and $y - b$ for fresh uniform a, b . Masked predicates reveal only $x - r$ for fresh uniform r . B2A and A2B conversion openings are one time padded by fresh masks. Local linear operations also do not reveal information about secret values. Sequentially composition of these facts gives the following privacy guarantee.

THEOREM 2. *Under fresh honest preprocessing, every fixed decapsulation circuit assembled from the MPC primitives given in Section 5 is perfectly simulatable against every static semi-honest coalition of fewer than t parties. That is, before prescribed output reconstruction, the corrupted classical view is identically distributed to a simulation from only the public inputs and the corrupted input and output interfaces. If the output is reconstructed, the reconstruction transcript is also simulatable from the reconstructed value.*

We note that the simulation here is information theoretic, and freshness is essential, because reusing a mask in two openings exposes the difference. Complete description of the simulators is given in Appendix B.1.

Table 3: Running time and communication cost per party of scheme A variants and scheme B.

t	N	Schem A: ML-KEM-512		Scheme A: ML-KEM-768		Scheme A: ML-KEM-1024		Scheme B	
		Median T_{dec} (ms)	C_{pp}^{out} (kB)	Median T_{dec} (ms)	C_{pp}^{out} (kB)	Median T_{dec} (ms)	C_{pp}^{out} (kB)	Median T_{dec} (ms)	C_{pp}^{out} (kB)
2	3	57	296	72	360	110	462	16	81
2	7	117	443	156	539	181	693	29	121
3	7	156	443	190	539	351	693	47	121
4	7	205	443	238	539	297	693	78	121
2	15	296	590	370	717	399	923	57	160
3	15	316	590	375	717	598	923	103	160
4	15	411	590	499	717	650	923	104	160
5	15	560	590	592	717	834	923	125	160
6	15	590	590	711	717	1022	923	152	160
7	15	683	590	779	717	1148	923	181	160
8	15	774	590	985	717	1148	923	191	160

8.3 Security of Scheme A

Since Scheme A's reconstructed output correctly recovers the output of ML-KEM decapsulation on every validated public key and ciphertext, we can combine this correctness with the MPC privacy in Theorem 2 to obtain the following composition reduction.

THEOREM 3 (SECURITY OF SCHEME A). *Assume that ML-KEM-512 is IND-CCA secure. In the static semi-honest, any TIND-CCA adversary $\mathcal{A}$ admits a reduction to KEM adversary $\mathcal{B}$ with almost the same running time of $\mathcal{A}$ and the additional cost of perfect transcript simulation. Moreover, it holds that*

$$\text{Adv}_A^{\text{TIND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{ML-KEM-512}}^{\text{IND-CCA}}(\mathcal{B}). \quad (73)$$

The reduction samples the corrupted key shares directly from their secret independent Shamir marginals, answers each threshold decapsulation query using its KEM decapsulation oracle, and uses the perfect MPC simulator for the corrupted transcript. See Appendix B.2.

8.4 Security of Scheme B

The underlying PKE and KEM of scheme B closely follow the design of Kyber [5]. This has the advantage that available security analysis of Kyber can be applied. First, we will reduce the CPA security of the PKE layer to Module LWE. Then, to prove KEM security in QROM, we use the Kyber style wrapper analysis in [23]. Appendix B.3.1 provides detailed analysis.

Since scheme B computes the underlying KEM decapsulation exactly on every validated public key and ciphertext, applying Theorem 2 gives TIND-CCA security of scheme B under IND-CCA security of its underlying KEM.

THEOREM 4 (SECURITY OF SCHEME B). *In the static semi-honest, any TIND-CCA adversary $\mathcal{A}$ on the TKEM scheme B admits a reduction to KEM adversary $\mathcal{B}$ with almost the same running time of $\mathcal{A}$ and the additional cost of perfect transcript simulation. Moreover, it holds that*

$$\text{Adv}_B^{\text{TIND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{B-KEM}}^{\text{IND-CCA}}(\mathcal{B}). \quad (74)$$

9 Evaluation

We have implemented our TKEM scheme A (all three variants based on ML-KEM-512/768/1024) and scheme B in C++. It is a simulator

for MPC protocols which runs computation of each party in the same idealised research prototype simulation environment. Our implementation also validated the implementations against their unthresholded KEM reference computations and the analytical communication cost.

9.0.1 Analytic Communication Complexity. The round complexity, public key and ciphertext sizes of our TKEM schemes are fixed, and do not depend on the threshold instance (t, N) . The communication package of each party has size logarithmic in the number of party N with analytical formulas given in Sections 6 and 7.

9.0.2 Threshold Decapsulation Running Time. We ran the experiments for t out of N decapsulation of scheme A variants and scheme B for (t, N) in the set $\{(2, 3), (2, 7), (3, 7), (4, 7), (2, 15), (3, 15), (4, 15), (5, 15), (6, 15), (7, 15), (8, 15)\}$. For benchmarking the running time of our threshold decapsulation protocols, we use the minimal qualified active set $s = |\mathcal{S}| = t$.

The simulator randomly chooses an active set of parties of size t . The Boolean field is selected by $\kappa = \min\{k \geq 1 : 2^k - 1 \geq N\}$. Thus, the implementation uses $\kappa = 2, 3, 4$ for $N = 3, 7, 15$, respectively.

We measured the running time T_{dec} which is the elapsed local time from entry into threshold decapsulation until the function returns a shared 256 bit session key. For each scheme and each (t, N) , we execute five repeat blocks. Each block performs five untimed warm-up decapsulations followed by twenty timed decapsulations. This give totally 100 runs for each experiment. We report the median of the 100 observations $\tilde{T} = \text{median}(T_1, \dots, T_{100})$.

We report the median decapsulation running time and concrete communication cost per party of our TKEM schemes in Table 3. The running time scales linearly in the threshold t for fixed N , and scales as $O(tN)$ in both variables t and N .

9.0.3 Preprocessing Cost. In the preprocessing phase, the threshold decapsulation protocols in scheme A for ML-KEM-512, 768, 1024 and scheme B requires generating 584703, 711679, 915455, 155391 Beaver triples, 6400, 7424, 9472, 6400 correlated B2A masks and 1024, 1280, 1536, 1024 arithmetic masked opening correlations respectively.

10 Conclusion

We study optimisation and relaxation for applying MPC methods to construct threshold decapsulation protocols for MK-KEM. Our first TKEM scheme A gives a threshold realisation of ML-KEM that preserves the standard ML-KEM's encapsulation and external interface. By exploiting the structure of decoding and validation predicates, parallelising independent pseudorandom function evaluations and rescheduling the decapsulation branches, we reduce the ML-KEM-512 decapsulation depth from a 384 round baseline MPC approach to 145 rounds in scheme A. The same construction extends to ML-KEM-768 and ML-KEM-1024. Our analysis also shows that after optimisation, the dominant source of interaction complexity is the secret input computation used for implicit rejection. Our scheme B redesigns the KEM around operations that are more suitable for MPC. Using uncompressed PKE and KT128 based hash functions, Scheme B reduces threshold decapsulation to 35 rounds. For both constructions, we provide security analyses and simulation implementations. It would be interesting to reduce the number of interaction rounds for fully ML-KEM-512 compatible TKEM to below 145 rounds.

References

- [1] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. 2025. *Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process*. NIST Internal Report NIST IR 8545. National Institute of Standards and Technology.
- [2] Donald Beaver. 1992. Efficient Multiparty Protocols Using Circuit Randomization. In *CRYPTO 1991*.
- [3] Rikke Bendlin and Ivan Damgard. 2010. Threshold Decryption and Zero-Knowledge Proofs for Lattice Based Cryptosystems. In *TCC 2010*.
- [4] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter Rasmussen, and Amit Sahai. 2018. Threshold Cryptosystems from Threshold Fully Homomorphic Encryption. In *CRYPTO 2018*.
- [5] Joppe W. Bos, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. 2018. CRYSTALS-Kyber: A CCA Secure Module Lattice Based KEM. In *EuroSP 2018*.
- [6] Katharina Boudgoust, Rafael del Pino, Oleksandra Lapiha, and Thomas Prest. 2026. Amber: a Family of Efficient Lattice Based IND-CCA Threshold KEMs. NIST Multi-Party Threshold Cryptography Project, Preview Writeup, Version 0.1.
- [7] Katharina Boudgoust, Rafael del Pino, Oleksandra Lapiha, and Thomas Prest. 2026. IND-CCA Lattice Threshold KEM Under 30 KiB. In *PKC 2026*.
- [8] Luis T. A. N. Brandao and Rene Peralta. 2026. *NIST First Call for Multi-Party Threshold Schemes*. NIST Interagency/Internal Report 8214C. National Institute of Standards and Technology.
- [9] Chris Brzuska, Michael Kloos, and Ivy Woo. 2026. Threshold Public-Key Encryption: Definitions, Relations, and CPA to CCA Transforms. In *PKC 2026*.
- [10] Valerio Cini, Russell Lai, and Ivy Woo. 2025. Pilvi: Lattice Threshold PKE with Small Decryption Shares and Improved Security. In *ASIACRYPT 2025*.
- [11] Cloudflare. 2026. *Keyless SSL Documentation*. Cloudflare, Inc. <https://developers.cloudflare.com/ssl/keyless-ssl/>
- [12] Kelong Cong, Daniele Cozzo, Varun Maram, and Nigel P. Smart. 2021. Gladius: LWR Based Efficient Hybrid Public Key Encryption with Distributed Decryption. In *ASIACRYPT 2021*.
- [13] Ivan Damgard, Valerio Pasto, Nigel Smart, and Sarah Zakarias. 2012. Multiparty Computation from Somewhat Homomorphic Encryption. In *CRYPTO 2012*.
- [14] Yvo Desmedt. 1993. Threshold Cryptosystems. In *AUSCRYPT 1992*.
- [15] Eiichiro Fujisaki and Tatsuaki Okamoto. 1999. Secure Integration of Asymmetric and Symmetric Encryption Schemes. In *CRYPTO 1999*.
- [16] Paul Grubbs, Varun Maram, and Kenneth Paterson. 2022. Anonymous, Robust Post-Quantum Public Key Encryption. In *EUROCRYPT 2022*.
- [17] Dennis Hofheinz, Kathrin Hovelmanns, and Eike Kiltz. 2017. A Modular Analysis of the Fujisaki-Okamoto Transformation. In *TCC 2017*.
- [18] Yisol Hwang, Shuichi Katsumata, Seonhong Min, Guilhem Niot, and Yongsoo Song. 2026. Proving Threshold Regev PKE from Adaptive Hint MLWE: Efficient, Noninteractive, and CCA Secure. Cryptology ePrint Archive, Paper 2026/1585.
- [19] Marcel Keller, Emmanuela Orsini, and Peter Scholl. 2016. MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer. In *ACM CCS 2016*.
- [20] Michael Kraitsberg, Yehuda Lindell, Valery Osheter, Nigel Smart, and Younes Talibi Alaoui. 2019. Adding Distributed Decryption and Key Generation to a Ring LWE Based CCA Encryption Scheme. In *ACISP 2019*.
- [21] Oleksandra Lapiha and Thomas Prest. 2025. A Lattice Based IND-CCA Threshold KEM from the BCHK+ Transform. In *ASIACRYPT 2025*.
- [22] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *EUROCRYPT 2010*.
- [23] Varun Maram and Keita Xagawa. 2023. Post-Quantum Anonymity of Kyber. In *PKC 2023*.
- [24] Daniele Micciancio and Adam Suhl. 2025. Simulation Secure Threshold PKE from LWE with Polynomial Modulus. *Communications in Cryptology* 1, 4 (2025).
- [25] National Institute of Standards and Technology. 2015. *SHA-3 Standard: Permutation Based Hash and Extendable Output Functions*. Federal Information Processing Standards Publication FIPS 202. U.S. Department of Commerce.
- [26] National Institute of Standards and Technology. 2024. *Module Lattice Based Key Encapsulation Mechanism Standard*. Federal Information Processing Standards Publication FIPS 203. U.S. Department of Commerce.
- [27] Oded Regev. 2005. On lattices, learning with errors, random linear codes, and cryptography. In *STOC 2005*.
- [28] Peter Schwabe, Douglas Stebila, and Thom Wiggers. 2020. Post-Quantum TLS Without Handshake Signatures. In *ACM CCS 2020*.
- [29] Peter Schwabe, Douglas Stebila, and Thom Wiggers. 2021. More Efficient Post-Quantum KEMTLS with Pre-Distributed Public Keys. In *ESORICS 2021*.
- [30] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613.
- [31] B. Viguier, D. Wong, G. Van Assche, Q. Dang, and J. Daemen. 2025. *KangarooTwelve and TurboSHAKE*. RFC 9861. RFC Editor.
- [32] Thom Wiggers, Sofia Celi, Peter Schwabe, Douglas Stebila, and Nick Sullivan. 2026. *KEM based Authentication for TLS 1.3*. Internet-Draft draft-celi-wiggers-tls-authkem-07. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/> Work in Progress.

Algorithm 4 Kyber PKE key generation for ML-KEM-512.

```

1: procedure K-PKE.KEYGEN( $d$ )
2:    $(\rho, \sigma) \leftarrow G(d \| \langle 2 \rangle_8)$ 
3:    $\tilde{A} \leftarrow \text{EXPAND}_A(\rho)$ 
4:    $N \leftarrow 0$ 
5:   for  $i \leftarrow 0$  to 1 do
6:      $s[i] \leftarrow \text{SamplePolyCBD}_3(\text{PRF}_3(\sigma, \langle N \rangle_8))$ 
7:      $N \leftarrow N + 1$ 
8:   end for
9:   for  $i \leftarrow 0$  to 1 do
10:     $e[i] \leftarrow \text{SamplePolyCBD}_3(\text{PRF}_3(\sigma, \langle N \rangle_8))$ 
11:     $N \leftarrow N + 1$ 
12:   end for
13:    $\tilde{s} \leftarrow \text{NTT}(s)$ 
14:    $\tilde{e} \leftarrow \text{NTT}(e)$ 
15:    $\tilde{t} \leftarrow \tilde{A} \circ \tilde{s} + \tilde{e}$ 
16:    $ek_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\tilde{t}) \| \rho$ 
17:    $dk_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\tilde{s})$ 
18:   return  $(ek_{\text{PKE}}, dk_{\text{PKE}})$ 
19: end procedure

```

A Additional Details on MPC Protocol

Section 5 defines the MPC primitives used by our TKEM constructions and gives their interaction and communication costs. This appendix provide additional details for the protocols.

A.1 Sharing and Correlated Openings

A.1.1 Evaluation Points. For arithmetic sharing, the N parties require distinct nonzero evaluation points in $\mathbb{F}_q$. With the natural choice of evaluation points $1, \dots, N$, we require $N < q$. For Boolean sharing over $\mathbb{F}_2^\kappa$, the corresponding condition is $\kappa = \lceil \log_2(N + 1) \rceil$. These conditions ensure that the Shamir sharings used in Section 5.1 are well defined. Reconstruction and all public linear operations then follow from the linearity of interpolation.

Table 4: Secret input and types of MPC evaluation of ML-KEM-512 operations.

Operation	Secret-dependent input	Type of MPC evaluation
ExpandA and parsing c, ek	none	public
DecLin	$\widehat{s}$	local on secret shares
DecodeMsg	w	nonlinear interval predicates
$G(m' h)$	m'	secret input SHA3-512
$J(z c)$	z	secret input SHAKE256
ExpandCoins(r')	r'	five secret input PRFs, CBD, and Boolean to arithmetic conversion
EncodeMsg	m'	Boolean to arithmetic conversion, and local scaling
ReEnclIn	m', y', e'_1, e'_2	local on arithmetic secret shares
CompEq	u', v'	nonlinear compressed coefficient predicates
Select	$b, K', K_\perp$	secret mux

A.1.2 Boolean to Arithmetic Conversion. Section 5.3.1 describes B2A using correlated sharings $[r]_2$ and $[r]_q$ of a uniform bit r . If $d = b \oplus r$ is opened, an equivalent algebraic expression for the arithmetic output is $[b]_q = [r]_q + d(1 - 2[r]_q)$. For $d = 0$, this returns $[r]_q$, while for $d = 1$ it returns $1 - [r]_q$. Because r is uniform and independent of b , the opened value d is itself uniformly random and reveals no information about b .

A.1.3 Masked Arithmetic Predicates. For the masked predicate of Section 5.3.2, let $r \xleftarrow{\$} \mathbb{Z}_q$ be independent of the secret coefficient x , and open $\delta = x - r \pmod{q}$. For every fixed x , the map $r \mapsto x - r$ is a permutation of $\mathbb{Z}_q$. Therefore δ is uniformly random and independent of x . Correctness follows from $x = r + \delta \pmod{q}$ which for every public $I \subseteq \mathbb{Z}_q$ gives $x \in I$ iff $r \in I - \delta$. Thus, the only opened arithmetic value is perfectly masked. After opening, the remaining computation is a Boolean predicate on preprocessed random bits and public endpoints.

A.2 ML-KEM Compression Preimages

For $D = 2^d$, we define the exact preimage of a compressed coefficient $a \in \{0, \dots, D - 1\}$ by $I_{a,d} := \{x \in \mathbb{Z}_q : \text{Compress}_d(x) = a\}$. FIPS 203 [26] compression is obtained by nearest integer rounding of the canonical representative $x \in \{0, \dots, q - 1\}$ followed by reduction modulo D . Before this final reduction, the scaled value is monotone in x . For output 0, the low and high ends may be joined by the final reduction modulo D . Thus, $I_{a,d}$ is a cyclic interval in $\mathbb{Z}_q$.

The interval endpoints depend only on the public parameters (q, d, a) which can be precomputed once by evaluating the standardised compression function on the $q = 3329$ possible coefficients. By writing the resulting cyclic interval as $I_{a,d} = [L_{a,d}, U_{a,d}] \pmod{q}$, the masked equality test used by Π_{CompEq} is $\text{Compress}_d(x) = a$ if and only if $r \in I_{a,d} - \delta$ and $\delta = x - r \pmod{q}$. The threshold protocols do not need to construct a secret compressed coefficient. Instead, they test membership in the exact preimage of the public compressed value.

A.2.1 Message Decoding. The ML-KEM message decoder is the special case $d = 1$ where $\text{Compress}_1(x) = 1$ iff $833 \leq x \leq 2496$. Therefore, Π_{Decode} is the masked interval test for $[833, 2496]$ which is applied independently to all 256 decrypted coefficients.

Algorithm 5 $\Pi_{\text{Decode}}([w]_q)$ for ML-KEM-512.

```

1: for  $i \leftarrow 0$  to 255 in parallel do
2:   Obtain correlated  $[r_i]_q$  and  $[\text{bits}_{12}(r_i)]_2$ 
3:    $[\delta_i]_q \leftarrow [w_i]_q - [r_i]_q$ 
4: end for
5: Open all  $\delta_i$  in one arithmetic opening round
6: for  $i \leftarrow 0$  to 255 in parallel do
7:    $(L_i, U_i) \leftarrow ([833, 2496] - \delta_i) \pmod{q}$ 
8:    $[m'_i]_2 \leftarrow \text{CYCLICINTERVAL}([r_i]_2, L_i, U_i)$ 
9: end for
10: return  $[m'_0]_2, \dots, [m'_{255}]_2$ 

```

A.2.2 Comparison. The balance comparison method in Section 5.3.3 associates to a bit segment the pair (e, g) , where e indicates equality with the corresponding public segment and g indicates that the secret segment is larger. If (e_H, g_H) and (e_L, g_L) denote the high and low segments, then $e = e_H e_L$, $g = g_H \oplus e_H g_L$. If the high segments differ, then $e_H = 0$ and $g = g_H$. If they agree, then $e_H = 1$ and the low segment decides the comparison. This gives correct comparison recursively.

A.3 MPC Protocols for Operations in ML-KEM

A.3.1 Π_{ExpandA} : Public Expansion and Parsing. All inputs to $\text{ExpandA}(\rho)$, ciphertext and public key parsings are public, the parties can locally run these operations. We call this local evaluation part Π_{ExpandA} which require no sharing, openings, or Beaver multiplication. The same applies to the public computation $H(ek)$, which can be cached with the threshold key.

A.3.2 Π_{DecLin} : Linear PKE Decryption. The ciphertext is publicly parsed into u and v , the parties hold $[\widehat{s}]_q$. They can locally compute

$$[w]_q = v - \text{NTT}^{-1}([\widehat{s}]_q^T \circ \text{NTT}(u)). \quad (75)$$

Correctness follows from the linearity of Shamir sharing and of the NTT maps. This part again does not require interaction.

A.3.3 Π_{Decode} : Message Decoding by Masked Intervals. For decoding message bits, we construction a protocol Π_{Decode} which applies the masked predicate check primitive to all 256 coefficients in one batch, and returns the Boolean shares of the resulting bits.

Its cost is $R(\Pi_{\text{Decode}}) = 6$, $M_2(\Pi_{\text{Decode}}) = 11520$, $O_q(\Pi_{\text{Decode}}) = 256$, $O_2(\Pi_{\text{Decode}}) = 0$. The output consists of 256 Boolean shared bits. The parties can locally group the bits into 32 logical message bytes.

A.3.4 Π_G : Threshold $G(m' || h)$ Evaluation. The parties concatenate the 256 secret message bits with a public sharing of h , giving 64-byte SHA3-512 input. Since SHA3-512 has 72-byte rate, domain suffix and padding can fit in the same sponge block, which requires only one Keccak- $f[1600]$ permutation [25].

In each Keccak round, θ , ρ , π , and the round constant addition are linear, the only nonlinear step is χ , see [25]. There is one secret multiplication for each of the 1600 state bits. All 1600 products in one Keccak round can be batch evaluated in one Beaver layer. Hence, one full Keccak- $f[1600]$ permutation has $R_{f[1600]} = 24$, $M_{2,f[1600]} = 38400$. The protocol Π_G outputs the first 256 digest bits as $[K']_2$ and the remaining 256 as $[r']_2$. Its cost is $R(\Pi_G) = 24$, $M_2(\Pi_G) = 38400$, $O_q = O_2 = 0$.

A.3.5 Π_J : Threshold Fallback Hashing. For ML-KEM-512 the input to J is $z || c$ of length 800 bytes. Since SHAKE256 has a 136-byte rate, the padded input occupies 6 absorption blocks. Each block is followed by a Keccak- $f[1600]$ permutation. The 32 output bytes fit within the state after the sixth permutation. The cost of Π_J is $R(\Pi_J) = 144$, $M_2(\Pi_J) = 230400$, $O_q(\Pi_J) = O_2(\Pi_J) = 0$.

A.3.6 Π_{Coins} : Threshold Pseudorandom Expansion and CBD. The parties hold the 256 Boolean-shared bits of r' . They append the public one-byte nonces 0, 1, 2, 3, 4 and evaluate the five SHAKE256 calls for coin expansion. Since every input has only 33 bytes, the input absorption fits in one SHAKE256 block.

The two $\eta_1 = 3$ calls request 192 output bytes which require two sequential permutations each. The three $\eta_2 = 2$ calls request 128 bytes which require one permutation each. Thus, the total work is seven permutations. However, we observe that the five calls have no data dependencies, hence can be evaluated in parallel. The resulting evaluation path requires the depth of only two permutations. Its cost is $R_{\text{PRF}}^{\text{par}} = 48$, $M_{2,\text{PRF}} = 268800$.

The five outputs contain 6144 secret bits. A single batched B2A converts these bits to $\mathbb{F}_q$ in one round. The CBD map is linear. For each coefficient and parameter η , we have

$$[\text{CBD}_\eta]_q = \sum_{j=0}^{\eta-1} [b_j]_q - \sum_{j=\eta}^{2\eta-1} [b_j]_q. \quad (76)$$

The total cost of thresholdising ExpandCoins is $R(\Pi_{\text{Coins}}) = 49$, $M_2(\Pi_{\text{Coins}}) = 268800$, $O_2(\Pi_{\text{Coins}}) = 6144$, $O_q(\Pi_{\text{Coins}}) = 0$.

A.3.7 Π_{Encode} : Message Encoding. The map EncodeMsg sends a bit m_i to $1665m_i \in \mathbb{F}_q$. To evaluate this, starting from $[m_i]_2$ the parties can run B2A, then multiply the resulting arithmetic sharing by the public constant 1665. Running in isolation, it costs $R(\Pi_{\text{Encode}}) = 1$, $M_2(\Pi_{\text{Encode}}) = 0$, $O_q(\Pi_{\text{Encode}}) = 0$, $O_2(\Pi_{\text{Encode}}) = 256$.

We will use batching to improve the cost over many runs. Both B2A calls use independent correlated masks, hence can be packed into the same opening layer which can convert 6400 Boolean bits in one round. The combined coin expansion and message encoding stage has 48 PRF rounds, followed by one shared conversion round.

A.3.8 Π_{ReEnc} : Reencryption. After Π_{Coins} and Π_{Encode} , the values y' , e'_1, e'_2 , and EncodeMsg(m') are already arithmetic shared. The parties locally evaluate

$$\begin{aligned} [u']_q &= \text{NTT}^{-1}(\widehat{A}^T \circ \text{NTT}([y']_q)) + [e'_1]_q, \\ [v']_q &= \text{NTT}^{-1}(\widehat{t}^T \circ \text{NTT}([y']_q)) + [e'_2]_q + [\text{EncodeMsg}(m')]_q. \end{aligned}$$

Algorithm 6 $\Pi_{\text{CompEq}}([u']_q, [v']_q, c)$.

```

1: Parse  $c$  into compressed targets  $a_u, a_v$ 
2: Flatten  $[u']_q, [v']_q$  into 768 arithmetic shared coefficients
3: for all 768 coefficients in parallel do
4:   Evaluate  $[e_j]_2 \leftarrow 1[\text{Compress}_{d_j}(x_j) = a_j]$ 
5: end for
6:  $[b]_2 \leftarrow \text{BALANCEDAND}([e_0]_2, \dots, [e_{767}]_2)$ 
7: return  $[b]_2$ 

```

which are linear in the secret shares, as $\widehat{A}$ and $\widehat{t}$ are public. Thus, this step does not require interaction.

A.3.9 Π_{CompEq} : Compressed Ciphertext Equality Check. The public ciphertext already contains the compressed target coefficients. Let $a_{i,j}$ ($a_{v,j}$) be the target for coefficient j of u_i (v) at width $d_u = 10$ ($d_v = 4$). The candidate ciphertext agrees with the public one if and only if $u'_i[j] \in I_{a_{i,j},10}$ for all $i \in \{0,1\}$ and $j \in \{0, \dots, 255\}$, and $v'[j] \in I_{a_{v,j},4}$ for all $j \in \{0, \dots, 255\}$. This requires 768 masked interval predicates, which can be evaluated in a single batch.

Each coefficient predicate consumes 45 Boolean multiplications. The final AND tree additionally consumes 767 ones. Its path depth is $\lceil \log_2 768 \rceil = 10$. Hence $R(\Pi_{\text{CompEq}}) = 16$, $M_2(\Pi_{\text{CompEq}}) = 35327$, $O_q(\Pi_{\text{CompEq}}) = 768$, $O_2(\Pi_{\text{CompEq}}) = 0$.

A.3.10 Π_{Mux} : Implicit Rejection and Secret Selection. Let $[b]_2$ be the ciphertext validity check bit, $[K']_2$ and $[K_\perp]_2$ be the two 256-bit key candidates. The parties evaluate the secret selection step bitwise

$$[K]_2 = [K_\perp]_2 \oplus [b]_2 ([K']_2 \oplus [K_\perp]_2). \quad (77)$$

The shared bit b is broadcast across the 256 lanes, after which all multiplications can be evaluated in one Beaver layer. The cost is $R(\Pi_{\text{Mux}}) = 1$, $M_2(\Pi_{\text{Mux}}) = 256$, $O_q = O_2 = 0$. The protocol outputs shares of the secret key. Key reconstruction requires an additional one round operation.

A.4 Canonical Equality for Scheme B

Scheme B uses uncompressed coefficients. Let $x \in \mathbb{Z}_q$ be secret and $a \in \{0, \dots, q-1\}$ be the public coefficient obtained from the canonical ciphertext encoding. Using the same preprocessed mask $r \xleftarrow{\$} \mathbb{Z}_q$, the parties open $\delta = x - r \pmod{q}$, and compute $\beta = a - \delta \pmod{q}$. Then $x = a$ iff $r = \beta$.

Because $r, \beta \in \{0, \dots, q-1\}$, equality is the AND of 12 bitwise XNOR values. The XNORs are local because β is public which can be implemented by a balanced AND tree uses 11 Boolean multiplications at depth 4. There are 768 ciphertext coefficients whose tests can be run in parallel, after which a balanced AND tree can combine their outputs. Hence, $R_B^{\text{Eq}} = 15$, $M_{2,B}^{\text{Eq}} = 9215$, $O_{q,B}^{\text{Eq}} = 768$.

Canonical decoding is essential for this equivalence. A public 12 bit string representing a value $\geq q$ is rejected before decapsulation. Every accepted ciphertext coefficient has a unique canonical representative in $\mathbb{Z}_q$.

A.5 Construction of MPC Protocols in Scheme B

The public values are $ek = (A, t)$, $c = (u, v)$, $h = H_{ek}(ek)$, $d = H_c(c)$. The secret elements are shared as $[s]_q$ and $[z]_2$. Specifically, each coefficient of s is Shamir secret shared over $\mathbb{F}_q$ and the 256 bits of z are shared over $\mathbb{F}_{2^\kappa}$.

Algorithm 7 Non-thresholded KEM underlying scheme B.

```

1: procedure SCHEMEB.KEYGEN
2:    $A \xleftarrow{\$} R_q^{2 \times 2}$ 
3:    $s, e \xleftarrow{\$} \chi_3^2$ 
4:    $t \leftarrow As + e$ 
5:    $z \xleftarrow{\$} \mathbb{B}^{32}$ 
6:    $ek \leftarrow (A, t)$ 
7:    $dk \leftarrow (ek, s, z)$ 
8:   return  $(ek, dk)$ 
9: end procedure

10: procedure SCHEMEB.ENCAPS( $ek$ )
11:    $m \xleftarrow{\$} \mathbb{B}^{32}$ 
12:    $h \leftarrow H_{ek}(ek)$ 
13:    $(\bar{K}, y, e_1, e_2) \leftarrow \text{BEXPAND}(m, h)$ 
14:    $c \leftarrow \text{BENC}(ek, m, y, e_1, e_2)$ 
15:    $d \leftarrow H_c(c)$ 
16:    $K \leftarrow \text{KT128}(\bar{K} \| d, C_g, 32)$ 
17:   return  $(K, c)$ 
18: end procedure

19: procedure SCHEMEB.DECAPS( $dk, c$ )
20:   Parse and canonically validate  $c = (u, v)$ 
21:    $(ek, s, z) \leftarrow dk$ 
22:    $h \leftarrow H_{ek}(ek)$ 
23:    $d \leftarrow H_c(c)$ 
24:    $w \leftarrow v - s^T u$ 
25:    $m' \leftarrow \text{DECODEMSG}(w)$ 
26:    $(\bar{K}', y', e'_1, e'_2) \leftarrow \text{BEXPAND}(m', h)$ 
27:    $c' \leftarrow \text{BENC}(ek, m', y', e'_1, e'_2)$ 
28:    $b \leftarrow 1[c' = c]$ 
29:    $K_{\text{good}} \leftarrow \text{KT128}(\bar{K}' \| d, C_g, 32)$ 
30:    $K_{\text{bad}} \leftarrow \text{KT128}(z \| d, C_b, 32)$ 
31:    $K \leftarrow \text{SELECT}(b, K_{\text{good}}, K_{\text{bad}})$ 
32:   return  $K$ 
33: end procedure

```

A.5.1 Decryption and Message Recovery. Since u and v are public, the parties can locally compute $[w]_q = v - [s]_q^T u$. Then, they can run the masked decoder protocol Π_{Decode} to obtain $[m']_2$. This costs $R_{\text{BDecode}} = 6$, $M_{2,\text{BDecode}} = 11520$, $O_{q,\text{BDecode}} = 256$.

A.5.2 Five KT128 Chunks. For the five KT128 calls in Eq. (52), the RFC 9861 single node input occupies less than one 168 byte TurboSHAKE128 rate block, and the output is at most 168 bytes. The only nonlinear work is one Keccak- $p[1600, 12]$ permutation for each chunk. The parties can Boolean share the state bits by using the shares of m' , then locally insert public suffix, customisation, length encoding, domain byte, and padding bit.

One Keccak round contributes one Boolean multiplication layer and 1600 Boolean multiplications through χ . Thus, a single KT128 chunk evaluation costs 12 rounds and 19200 Boolean multiplications. The five chunks are mutually independent, hence can be evaluated in parallel. It costs $R_{\text{BExpand}} = 12$ and $M_{2,\text{BExpand}} = 96000$. Their concatenated Boolean output is parsed locally into $[\bar{K}']_2$ and 6144 Boolean coin bits $[\omega']_2$.

A.5.3 Sampling and Message Conversion. The ciphertext validation branch requires arithmetic shares of CBD sampled bits and 256 recovered message bits. We concatenate $[\omega']_2 \| [m']_2 \in \mathbb{F}_{2^\kappa}^{6400}$, then apply one batched Boolean to arithmetic conversion which costs $R_{\text{B2A}}^B = 1$ and $O_2^B = 6400$. After this, CBD evaluation is linear as each centered binomial coefficient is a signed sum of converted bits. $\mu(m')$ is obtained by multiplying each converted message bit by 1665. The parties can now derive $[y']_q, [e'_1]_q, [e'_2]_q, [\mu(m')]_q$.

Table 5: MPC Protocols of Operations in scheme B and their cost.

Protocol	R	M_2	O_q	O_2
Public parse, H_{ek}, H_c	0	0	0	0
BDecLin	0	0	0	0
Π_{Decode}	6	11520	256	0
Π_{BExpand}	12	96000	0	0
Fused B2A	1	0	0	6400
BReEnc	0	0	0	0
Π_{BEq}	15	9215	768	0
Π_{BKDF}	12	38400	0	0
Π_{Mux}	1	256	0	0

A.5.4 Reencryption. Using public A, t , the parties can locally compute $[u']_q = A^T [y']_q + [e'_1]_q$ and $[v']_q = t^T [y']_q + [e'_2]_q + [\mu(m')]_q$. Compared to scheme A, this step saves the compression evaluations.

A.5.5 Masked Equality Checks. There are 768 polynomial coefficients in (u', v') . Let $[x]_q$ be a coefficient and $a \in \{0, \dots, q-1\}$ be the corresponding coefficient of the public input ciphertext. Preprocessing supplies a uniform $r \xleftarrow{\$} \mathbb{F}_q$ with both an arithmetic sharing $[r]_q$ and a Boolean sharing of its canonical 12-bit representation.

The parties open $\delta = x - r \pmod{q}$, then define the public value $\beta = a - \delta \pmod{q}$. We have $x = a$ iff $r = \beta \pmod{q}$ which is an equality check of a secret 12 bit element with a public 12 bit constant. XNOR with the public bits is local, and a balanced AND tree check of the 12 equality bits requires 11 Boolean multiplications with depth 4.

All 768 masked openings can be batched into one arithmetic opening round, and all coefficient equality tree checks can be run in parallel. A final balanced AND tree check over the resulting 768 bits has depth 10 which uses 767 additional Boolean multiplications.

The whole ciphertext equality check protocol cost $R_{\text{BEq}} = 15$, $M_{2,\text{BEq}} = 9215$, $O_{q,\text{BEq}} = 768$. The result for each party is a Boolean sharing $[b]_2$ satisfying $b = 1$ if and only if $c' = c$.

A.5.6 Key Derivation. When $[\bar{K}']_2$ is available, the parties evaluate the two secret KT128 calls

$$[K_{\text{good}}]_2 = \text{KT128}([\bar{K}']_2 \| d, C_g, 32), \quad (78)$$

$$[K_{\text{bad}}]_2 = \text{KT128}([z]_2 \| d, C_b, 32). \quad (79)$$

For each call, the secret input is 32 bytes and the public ciphertext digest d contributes another 32 bytes. The customisation strings have 18 bytes for C_g and 17 bytes for C_b , and both corresponding length encodings occupy 2 bytes. Thus, their complete RFC 9861 single node inputs have lengths 84 and 83 bytes, respectively, which are below the 168 byte rate. Each requested output is 32 bytes. Each call needs only one Keccak- $p[1600, 12]$ permutation. The two calls can be run in parallel, which costs $R_{\text{BKDF}} = 12$, $M_{2,\text{BKDF}} = 38400$. Together with the five expansion KT128 calls, scheme B evaluation 7 KT128 permutations.

A.5.7 Threshold Decapsulation Protocol. The complete threshold decapsulation protocol in scheme B is presented in Algorithm 3.

We now count the circuit depth. Message decoding step costs 6 rounds which must precede the expansion step. Then, the five chunks contribute 12 rounds. After expansion, the validation branch has depth $R_{\text{validation}}^B = 16$, whereas the key derivation branch has depth $R_{\text{KDF}}^B = 12$. Thus, the ciphertext validation branch dominates circuit depth which gives $R_B^{\text{dec}} = 35$. The parties hold a secret shared 256 bit key. If the session key will be opened to the designated recipient, one additional batched reconstruction round is needed $R_B^{\text{out}} = 35$.

Using the packing messages for communication, we can compute the total communication cost as $C_B^{\text{dec}} = s(317182\kappa + 12 \cdot 1024)$ bytes, and with key reconstruction $C_B^{\text{dec}} = s((317182 + 156)\kappa + 12 \cdot 1024)$ bytes.

A.5.8 Correctness. We show that reconstruction from scheme B output key shares recovers the decapsulation result correctly. This can be seen by checking step by step.

The public values $ek, c, h = H_{ek}(ek)$, and $d = H_c(c)$ are identical in the two executions. By linearity of Shamir sharing, linear decryption reconstructs exactly $v - s^T u$. output of Π_{Decode} can be reconstructed to m' .

For the five threshold KT128 evaluations, reconstruction also gives the correct strings Y_j , hence, the correct $\bar{K}'$ and ω' . B2A preserves the raw bits in $\mathbb{F}_q$. CBD and message embedding are linear in these arithmetic bits.

Therefore, reencryption output can reconstruct

$$c' = \text{BEnc}(ek, m', y', e'_1, e'_2). \quad (80)$$

For each coefficient, equality check is an algebraic equivalence over $\mathbb{F}_q$. The final AND reconstructs to $b = 1$ iff when the coefficients are equal.

Moreover, the two threshold key derivation calls reconstruct the keys. Π_{Mux} returns the good/bad key when $b = 1/0$. This is the corresponding output of the KEM in Algorithm 7.

A.6 Communication Cost

Recall from that for a given protocol, M_2, O_2 and O_q are the numbers of Boolean multiplications, Boolean openings and arithmetic openings. Each active party has online communication cost $C_{\text{pp}} = (2M_2 + O_2)\kappa + 12O_q$ bits.

A.6.1 End to End Cost of Scheme A-512. Summing the number of Boolean multiplications of the six interactive subprotocols in scheme A-512 gives $M_{2,A512} = 584703$. The batched conversion of the 6144 CBD bits and 256 message bits gives $O_{2,A512} = 6400$, while decoding and compressed equality contribute $O_{q,A512} = 1024$. Thus, the per party end to end communication cost of scheme A-512 is $C_{A512,\text{pp}}^{\text{out}} = 1176062\kappa + 12288$ bits.

A.6.2 End to End Communication Cost of Scheme B. Scheme B has $M_{2,B} = 155391, O_{2,B} = 6400, O_{q,B} = 1024$. Hence, the per party communication cost of scheme B is $C_{B,\text{pp}}^{\text{out}} = 317438\kappa + 12288$ bits.

A.6.3 Preprocessing Cost. The input independent preprocessing cost for each decapsulation call is $\text{Prep}_{A512} = (\text{Beaver} = 584703, B2AM = 6400, aM = 1024)$ and $\text{Prep}_B = (\text{Beaver} = 155391, B2AM = 6400, aM = 1024)$ where records *Beaver* is the number of Boolean

Beaver triples, *B2AM* is the number of Boolean to Arithmetic correlated masks, and *aM* is the number of arithmetic masked predicate correlations, respectively.

B Security Proofs

This appendix gives the detailed security arguments for our TKEM scheme A and B. The proofs are modular. We first explain the information theoretic privacy of the MPC layer. Security of Scheme A then follows by composition with security of ML-KEM. For Scheme B, we need to analyse the underlying PKE, its decryption failure probability, deterministic encryption induced by the random oracle derived coins, and Kyber style QROM wrapper.

B.1 Passive Security of the MPC Layer

B.1.1 Model and view. We fix a set of N parties $P_1, \dots, P_N$, a reconstruction threshold t , and a static corruption set $C \subseteq [N]$ with $|C| < t$. Arithmetic values are shared by degree $(t-1)$ Shamir sharing over $\mathbb{F}_q$, and Boolean values by degree $(t-1)$ Shamir sharing over $\mathbb{F}_{2^\kappa}$. We assume $N < q$ and $2^\kappa - 1 \geq N$, so that both sharing domains contain sufficient distinct nonzero evaluation points.

The trusted preprocessing dealer correctly generates prescribed correlations. In particular, each invocation receives fresh one-time use Beaver triples, fresh arithmetic and Boolean masks, cross domain representations, and any zero sharings used to refresh an output. Different sharings of the same underlying mask use independent random Shamir polynomials in their respective fields. The online protocol uses authenticated and reliable communication together with a consistent opening operation. Maliciously inconsistent broadcasts are outside the scope of our analysis.

For a protocol execution Π , the view of the corrupted parties includes their input and preprocessing shares, local randomness, received messages, publicly reconstructed masked values, and output shares. Let $\mathbb{F}$ be either $\mathbb{F}_q$ or $\mathbb{F}_{2^\kappa}$.

LEMMA 5. *For $x \in \mathbb{F}$, let $[x]$ be a degree $(t-1)$ Shamir sharing obtained by choosing its nonconstant coefficients uniformly random in $\mathbb{F}$. For every fixed coalition C with $|C| < t$, the vector $([x]_i)_{i \in C}$ is uniformly random over $\mathbb{F}^{|C|}$, and is independent of x . The same holds for for any collection of sharings whose sharing polynomials are sampled independently, even when the underlying secrets are related.*

PROOF. This is a basic property of Shamir secret sharing [30]. We write the sharing polynomial as

$$f(X) = x + a_1X + \dots + a_{t-1}X^{t-1}. \quad (81)$$

For $c = |C| < t$, evaluation at the c distinct nonzero points belonging to C is an affine map from $(a_1, \dots, a_{t-1})$ onto $\mathbb{F}^c$. Its linear part contains a $c \times c$ Vandermonde minor of full rank. Hence, every vector of corrupted shares has exactly $|\mathbb{F}|^{t-1-c}$ preimages which occurs with probability $|\mathbb{F}|^{-c}$, independently of the constant term x . Independent sharing polynomials give the joint distribution statement by taking products of these uniformly random marginals. $\square$

B.1.2 Simulation of MPC Protocols. We first recall the basic privacy property of Beaver and masked openings.

LEMMA 6 (BEAVER OPENINGS). *Let $\langle a, b, c \rangle$ be a fresh Beaver triple with $a, b \xleftarrow{\$} \mathbb{F}$ independently and $c = ab$. In the multiplication of*

shared values $[x]$ and $[y]$, the publicly opened values $d = x - a, e = y - b$ are jointly uniformly random in $\mathbb{F}^2$, and independent of (x, y) . Conditioned on (d, e) and the corrupted preprocessing shares, the corrupted parties' updated output shares can be computed locally from their simulated input and triple shares.

LEMMA 7 (MASKED OPENINGS). *The two masking patterns used in this work are perfectly private. Specifically, we have*

- (1) If $r \xleftarrow{\$} \mathbb{F}_q$ is fresh and independent, then $\delta = x - r$ is uniformly random in $\mathbb{F}_q$ and independent of x .
- (2) If $r \xleftarrow{\$} \{0, 1\}$ is fresh and independent, then $d = b \oplus r$ is uniformly random in $\{0, 1\}$ and independent of b .

The statements remain true in the joint view of fewer than t corrupted parties when the arithmetic and Boolean representations of r are shared with independent Shamir polynomials.

The Boolean to arithmetic conversion is an instance of Lemma 7. After opening $d = b \oplus r$, the arithmetic share is obtained from the already shared $[r]_q$ by the public affine identity $[b]_q = [r]_q + d(1 - 2[r]_q)$. Dealer assisted arithmetic to Boolean conversion opens a freshly masked arithmetic difference, and then evaluates only public shifts and Boolean circuit. Thus, its privacy is also covered by the same argument.

THEOREM 8 (PERFECT PASSIVE PRIVACY OF MPC CIRCUIT). *Let f be a deterministic function evaluated by a fixed circuit whose online operations are compositions of the following primitives:*

- (1) local linear operations on Shamir shares;
- (2) multiplication by public constants and public ring elements;
- (3) Beaver multiplication using fresh triples;
- (4) dealer assisted B2A and A2B conversions using fresh masks;
- (5) openings of values masked by fresh uniform arithmetic or Boolean masks;
- (6) fixed Boolean circuits whose AND gates are evaluated by fresh Beaver triples;
- (7) local output refresh by adding a fresh preprocessing sharing of zero.

Then, against every static semi-honest corruption set C with $|C| < t$, the online view before any prescribed final output reconstruction is perfectly simulatable from only the public inputs and the corrupted parties' prescribed input and output interfaces. Equivalently, the real and simulated classical views are identically distributed.

PROOF. The simulator first samples the corrupted parties' preprocessing shares from their exact subthreshold marginals. By Lemma 5, these marginals are uniform and do not require knowing the hidden preprocessing secrets.

Local linear operations generate no messages and are reproduced directly on the simulated shares. At a Beaver multiplication, the simulator samples the two public openings d, e as independent uniformly random field elements and updates the corrupted output shares accordingly. Lemma 6 shows that this is exactly the real conditional distribution. At a masked arithmetic predicate or conversion, the simulator samples the opened mask difference uniformly and uses the public post processing rule. Lemma 7 again gives equality of distributions. Boolean AND layers are Beaver multiplications in $\mathbb{F}_{2^k}$, so the same simulation applies at each gate and can be batched as in the real protocol.

Because every preprocessing item is fresh and used once, the simulator's choices for distinct interactions have the same independence relations as the real execution. An induction over the circuit establishes equality of the entire corrupted view. The argument is information theoretic with no computational assumption. $\square$

We emphasise that freshness is a crucial assumption in our model. Reusing the same arithmetic mask r in two openings would reveal $(x - r) - (x' - r) = x - x'$. Similar issues apply to reused bit masks and Beaver triples. The ideal preprocessing model here needs to provide fresh one-time use correlations for every decapsulation call.

B.2 Security of Scheme A

B.2.1 *Correctness.* Assume that the preprocessing phase is correct and the active set S has $|S| \geq t$. Any reconstructed session key of Scheme A's output equals the decapsulated key of ML-KEM-512 on the same validated ciphertext. Let us check the output of each operation in scheme A one by one.

- (1) Ciphertext and public key parsing, decompression, ExpandA, NTT operations, and multiplication by public ring elements are deterministic public or linear operations. Therefore, Π_{DecLin} reconstructs $w = \text{DecLin}(dk_{\text{PKE}}, c)$.
- (2) For each coefficient of w , the masked interval protocol returns 1 precisely when the canonical representative lies in $[833, 2496]$. Therefore, the reconstructed 256 bits are exactly $m' = \text{DecodeMsg}(w)$.
- (3) Π_G evaluates the same Keccak- $f[1600]$ permutation, SHA3-512 padding and domain separation as in ML-KEM-512. It reconstructs $(K', r') = G(m' || h)$. The parallel fallback protocol reconstructs $K_\perp = J(z || c)$. We note that parallel scheduling does not change either value because the branches have no data dependency.
- (4) The five PRF evaluations in Π_{Coins} use the same r' , one-byte nonces, SHAKE256 domain separation, output lengths, and CBD maps. Boolean to arithmetic conversion preserves each bit value, and Π_{Encode} maps each reconstructed message bit to 0 or 1665. Hence, the reconstructed arithmetic values are exactly the values y', e'_1, e'_2 and $\text{EncodeMsg}(m')$ in ML-KEM-512.
- (5) Every remaining ring operation in Π_{ReEnc} is linear with public coefficient. Therefore, reconstruction therefore gives

$$(u', v') = \text{ReEncLin}(ek, m', y', e'_1, e'_2). \quad (82)$$

- (6) For each of the 768 candidate coefficients, Π_{CompEq} tests membership in the exact preimage interval of the corresponding public compressed coefficient. Therefore, its reconstructed bit satisfies $b = 1$ iff $\text{PackCT}(u', v') = c$.
- (7) Finally, Π_{MUX} reconstructs $\text{Select}(b, K', K_\perp)$.

Thus each MPC protocol in scheme A has the same output as the corresponding operation in ML-KEM-512.

B.2.2 *TIND-CCA Security.* Scheme A's protocol correctly evaluates ML-KEM decapsulation function. In the static semi-honest TIND-CCA experiment, an adversary $\mathcal{A}$ chooses a corruption set C with $|C| < t$ before seeing the threshold shares. It receives the

public encapsulation key, the corrupted key shares and preprocessing shares. A threshold decapsulation query on ciphertext c returns the same external key that a legitimate caller would receive, and exposes the complete corrupted party view of this MPC execution.

At challenge time the challenger runs ordinary encapsulation, returns the challenge ciphertext together with either its real key or an independent uniform key, and disallows post challenge decapsulation query on the challenge ciphertext. We write $\text{Adv}_A^{\text{TIND-CCA}}(\mathcal{A})$ for the resulting distinguishing advantage.

THEOREM 9. *Assume IND-CCA KEM security of ML-KEM-512. Under the trusted fresh preprocessing model of Theorem 8, there is a reduction to KEM adversary $\mathcal{B}$ for any TIND-CCA TKEM adversary $\mathcal{A}$ with essentially the running time and additional cost of perfect transcript simulation, which satisfies*

$$\text{Adv}_A^{\text{TIND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{ML-KEM-512}}^{\text{IND-CCA}}(\mathcal{B}) \quad (83)$$

PROOF. Let $\mathcal{B}$ participate in the IND-CCA KEM experiment and forwards the public ML-KEM encapsulation key to $\mathcal{A}$. Note that $\mathcal{B}$ does not know the whole decapsulation key. For every fixed hidden key, fewer than t Shamir shares of each secret component have a distribution independent of that component. Hence, $\mathcal{B}$ samples the corrupted shares of $\hat{s}$ and z directly from their exact uniform marginals. This also allows it to sample the corrupted portions of the trusted preprocessing.

For a decapsulation query c , $\mathcal{B}$ asks its KEM decapsulation oracle for $K = \text{ML-KEM-512.Decaps}(dk, c)$, which is the value reconstructed by Scheme A. Theorem 8 then allows $\mathcal{B}$ to generate a perfect corrupted party transcript for that invocation. If output shares themselves are exposed, preprocessing refresh makes their subthreshold marginal uniformly random. If the output is reconstructed, $\mathcal{B}$ simulates the reconstruction from the known K .

The challenge ciphertext and challenge key are forwarded exactly from the KEM security experiment. Thus, every part of $\mathcal{A}$'s view is identically distributed to its threshold experiment, so that any distinguishing advantage transfers without loss. $\square$

The same proof applies to ML-KEM-768 and ML-KEM-1024 variants of scheme A.

B.3 Security of Scheme B

B.3.1 Scheme-B's PKE. We first recall the MLWE problem. For distributions χ_s and χ_e , let $\text{MLWE}_{m,k}^{\chi_s, \chi_e}$ denote the problem of distinguishing $(A, Ax + e)$ from (A, u) where $A \xleftarrow{\$} R_q^{m \times k}$, $x \xleftarrow{\$} \chi_s^k$, $e \xleftarrow{\$} \chi_e^m$, and $u \xleftarrow{\$} R_q^m$.

For a fixed message m , we define randomised ciphertext pseudorandomness as distinguishing the real Scheme B public key together with $c \leftarrow \text{BEnc}(ek, m; y, e_1, e_2)$ from the same public key together with an independent uniformly sampled canonical element of R_q^3 . We allow the adversary to choose m after seeing the public key.

LEMMA 10. *Let ϵ_{pk} denote the advantage of the MLWE reductions needed to replace a real Scheme B public key vector $\mathbf{t}$ by uniformly random vector, and ϵ_{ct} denote the advantage of the ciphertext MLWE*

reduction $\epsilon_{\text{pk}} = \text{Adv}_{\text{MLWE}_{2,2}^{\chi_3, \chi_3}}^{\text{MLWE}_{2,2}^{\chi_3, \chi_3}}$, $\epsilon_{\text{ct}} = \text{Adv}_{\text{MLWE}_{3,2}^{\chi_3, \chi_2}}^{\text{MLWE}_{3,2}^{\chi_3, \chi_2}}$. Then encryption of every fixed or adversarially selected message satisfies

$$\epsilon_{\text{rpr}} := \text{Adv}_{\text{PKE}_B}^{\text{RPR}} \leq 2\epsilon_{\text{pk}} + \epsilon_{\text{ct}}. \quad (84)$$

Moreover, for any polynomial time distinguishers $\mathcal{B}_1, \mathcal{B}_2$,

$$\text{Adv}_{\text{PKE}_B}^{\text{IND-CPA}} \leq \text{Adv}_{\text{MLWE}_{2,2}^{\chi_3, \chi_3}}^{\text{MLWE}_{2,2}^{\chi_3, \chi_3}}(\mathcal{B}_1) + \text{Adv}_{\text{MLWE}_{3,2}^{\chi_3, \chi_2}}^{\text{MLWE}_{3,2}^{\chi_3, \chi_2}}(\mathcal{B}_2). \quad (85)$$

PROOF. For ciphertext pseudorandomness, Game 0 contains the real public key $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, $\mathbf{s}, \mathbf{e} \xleftarrow{\$} \chi_3^2$, and a real randomised encryption of the selected message. Game 1 replaces $\mathbf{t}$ by a uniform element of R_q^2 . This costs at most ϵ_{pk} .

Conditioned on Game 1, define $\tilde{\mathbf{A}} := \begin{pmatrix} \mathbf{A}^T \\ \mathbf{t}^T \end{pmatrix} \in R_q^{3 \times 2}$. Because $\mathbf{A}$ and $\mathbf{t}$ are independent and uniform, so is $\tilde{\mathbf{A}}$. One has

$$\begin{pmatrix} \mathbf{u} \\ v - \mu(m) \end{pmatrix} = \tilde{\mathbf{A}}\mathbf{y} + \begin{pmatrix} \mathbf{e}_1 \\ e_2 \end{pmatrix}, \quad (86)$$

where $\mathbf{y} \xleftarrow{\$} \chi_3^2$ and $(\mathbf{e}_1, e_2) \xleftarrow{\$} \chi_2^3$.

Game 2 replaces this vector by a uniformly random element of R_q^3 at cost of at most ϵ_{ct} . Adding $(0, 0, \mu(m))$ is a permutation of R_q^3 , so the complete ciphertext is uniform.

The target pseudorandomness experiment keeps the public key distributed as in real key generation. The final hybrid replaces the uniform $\mathbf{t}$ in Game 2 by a real $\mathbf{A}\mathbf{s} + \mathbf{e}$ while leaving the already uniformly random ciphertext independent. Reversing the first MLWE hybrid costs another ϵ_{pk} . This gives Eq. (84).

For IND-CPA security, we do not need to restore the public key distribution. We can run the first two hybrids inside the IND-CPA game, which itself chooses the challenge bit. After Game 2, the challenge ciphertext is uniformly random and independent of that bit, so the adversary has no advantage in the final game. The two game differences are bounded by one $\text{MLWE}_{2,2}^{\chi_3, \chi_3}$ reduction and one $\text{MLWE}_{3,2}^{\chi_3, \chi_2}$ reduction, which proves Eq. (85). $\square$

B.3.2 Decryption Failure Probability. The Kyber style QROM analysis of [23] requires δ correctness in which the probability is averaged over key generation but maximised over the message. For a fixed PKE key pair, we define

$$p_{\text{fail}}(ek, sk; m) := \Pr_r[\text{PKE}_B.\text{Dec}(sk, \text{PKE}_B.\text{Enc}(ek, m; r)) \neq m]. \quad (87)$$

The correctness parameter is required to satisfy

$$\mathbb{E}_{(ek, sk) \leftarrow \text{PKE}_B.\text{KeyGen}} \left[\max_{m \in \{0,1\}^{256}} p_{\text{fail}}(ek, sk; m) \right] \leq \delta_B. \quad (88)$$

We now give an explicit bound satisfying Eq. (88).

For $X \leftarrow \text{CBD}_\eta$, the exact one coefficient distribution is

$$p_\eta(j) := \Pr[X = j] = 4^{-\eta} \binom{2\eta}{\eta+j}, \quad -\eta \leq j \leq \eta \quad (89)$$

whose moment generating function is

$$M_\eta(\theta) := \mathbb{E}[e^{\theta X}] = \cosh(\theta/2)^{2\eta}. \quad (90)$$

For independent $X \leftarrow \text{CBD}_a$ and $Y \leftarrow \text{CBD}_b$, we define

$$M_{a,b}(\theta) := \mathbb{E}[e^{\theta XY}] = \sum_{i=-a}^a \sum_{j=-b}^b p_a(i) p_b(j) e^{\theta ij}. \quad (91)$$

We fix one coefficient index of the integer lift of the noise polynomial

$$v = \mathbf{e}^T \mathbf{y} + e_2 - \mathbf{s}^T \mathbf{e}_1 \quad (92)$$

Each of the two coordinates of $\mathbf{e}^T \mathbf{y}$ contributes 256 products of independent CBD_3 variables, which gives 512 products in total. Similarly, $\mathbf{s}^T \mathbf{e}_1$ contributes 512 independent $\text{CBD}_3 \cdot \text{CBD}_2$ products, and e_2 contributes one CBD_2 variable.

For a fixed output coefficient, every sampled input coefficient appears exactly once in the corresponding sum. Therefore, we have

$$\mathbb{E}[e^{\theta v_i}] = M_{3,3}(\theta)^{512} M_{3,2}(\theta)^{512} M_2(\theta). \quad (93)$$

The distribution of v_i is symmetric, so for every $\theta > 0$, Chernoff's inequality gives

$$\Pr[|v_i| \geq 832] \leq 2e^{-832\theta} M_{3,3}(\theta)^{512} M_{3,2}(\theta)^{512} M_2(\theta). \quad (94)$$

It remains to relate this tail event to the maximisation over messages in Eq. (88). For every fixed secret key and every message, if all integer lifted coefficients satisfy $|v_i| \leq 831$, then a coefficient encoding message bit 0 lies outside the decoding interval [833, 2496], while a coefficient encoding message bit 1 lies inside that interval. Hence, every decryption failure is contained in the message-independent event $\mathcal{E} := \{\exists i \in \{0, \dots, 255\} : |v_i| \geq 832\}$. Consequently, for each fixed key it holds that

$$\max_m p_{\text{fail}}(ek, sk; m) \leq \Pr[\mathcal{E} \mid ek, sk]. \quad (95)$$

Taking expectation over key generation and then applying a union bound over the 256 coefficients yields

$$\delta_B \leq 256 \cdot 2e^{-832\theta} M_{3,3}(\theta)^{512} M_{3,2}(\theta)^{512} M_2(\theta). \quad (96)$$

No independence between different output coefficients is assumed for this final union bound.

At $\theta = 0.36$, direct evaluation of the exact finite sums gives $M_{3,3}(0.36) < 1.17$, $M_{3,2}(0.36) < 1.10$, $M_2(0.36) < 1.06$. Substitution into Eq. (94) gives a per coefficient bound below $2^{-235.54}$. Counting the factor $256 = 2^8$ in Eq. (96), it gives $\delta_B < 2^{-227.54} < 2^{-227.5}$.

B.3.3 QROM Specification of Hash Calls. In scheme B, we do not write the FO transform as a single $G(m)$ and $H(m, c)$. Like Kyber, it hashes the public key into a public context, derives a pre-key together with encryption coins, then hashes the ciphertext, and finally hashes the pre-key and ciphertext digest. The work [16] shows that pre-key and nested hash layers prevents applying a black-box FO transform theorem without additional analysis. The work [23] subsequently gave a wrapper based QROM analysis that handles this Kyber syntax directly and yields an explicit IND-CCA bound. We will apply this technique to analyse scheme B.

Expansion oracle. We model each customisation string domain of KT128 in scheme B as a quantum random oracle. For fixed length m and h , the five inputs $m||h||\langle 0 \rangle_8, \dots, m||h||\langle 4 \rangle_8$ are pairwise distinct and injectively encoded. In random oracle model, their outputs are independent uniformly random strings. We define the oracle

$$G^*(m, h) := Y_0 || Y_1 || Y_2 || Y_3 || Y_4 = \bar{K} || r, \quad (97)$$

where $r \in \{0, 1\}^{6144}$ is the coin string. As a distribution over functions, G^* is a random function with an 800 byte output. Granting an adversary one query that returns the full tuple only strengthens

access relative to asking separately for one component. This is because every real component query can be simulated with one full tuple query and discarding the unused chunks.

Public digests. The customisation strings C_{ek} and C_c are represented as one random oracle

$$H(pk, ek) := H_{ek}(ek), \quad H(ct, c) := H_c(c). \quad (98)$$

Because the tags are injective and disjoint, this is distributionally identical to two independent random oracle domains. We count all quantum queries to these two restrictions as q_H .

Good and fallback KDFs. Similarly, we define the oracle

$$H'(\text{good}, x, d) := \text{KT128}(x || d, C_g, 32), \quad (99)$$

$$H'(\text{bad}, x, d) := \text{KT128}(x || d, C_b, 32). \quad (100)$$

The branch tag makes the two restrictions independent random functions in the ideal model. Let $q_{H'}$ be the total quantum queries across both tagged restrictions. All oracle inputs remain injectively encoded. The tags can only separate inputs that would otherwise share a domain. Thus, the same collision and programming bounds apply with q_H and $q_{H'}$ interpreted as these total tagged domain query counts.

Under this notation Scheme B's KEM decapsulation has the form

$$h = H(pk, ek), \quad (101)$$

$$(\bar{K}, r) = G^*(m, h), \quad (102)$$

$$c = \text{BEnc}(ek, m; r), \quad (103)$$

$$d = H(ct, c), \quad (104)$$

$$K_{\text{good}} = H'(\text{good}, \bar{K}, d), \quad (105)$$

$$K_{\text{bad}} = H'(\text{bad}, z, d). \quad (106)$$

Decapsulation decrypts m' , recomputes $(\bar{K}', r') = G^*(m', h)$ and $c' = \text{BEnc}(ek, m'; r')$, returns the good value when $c' = c$, and otherwise returns the fallback value. This is the Kyber style implicitly rejecting wrapper analysed in [23], up to the explicit injective domain tags.

B.3.4 IND-CCA Security in the QROM of Scheme B's KEM. We can now use the δ correctness in Eq. (88) to prove a concrete QROM IND-CCA bound following the analysis of [23].

THEOREM 11. *We model the KT128 hash calls as the quantum random oracles G^* , H , and H' defined above. Assume that $\mathcal{A}$ make at most q_D classical decapsulation queries and at most $q_G, q_H, q_{H'}$ quantum queries to the three oracles, respectively. We define $\epsilon_{\text{CPA}, B} := \text{Adv}_{\text{PKE}_B}^{\text{IND-CPA}}(\mathcal{B})$. Then, there exists an IND-CPA adversary $\mathcal{B}$ against the Scheme B PKE such that*

$$\begin{aligned} \text{Adv}_B^{\text{IND-CCA}}(\mathcal{A}) &\leq 2(q_G + q_{H'})\sqrt{\epsilon_{\text{CPA}, B} + 2^{-256}} \\ &\quad + \frac{9q_{H'} + 2q_H}{2^{128}} + 4q_G\sqrt{\delta_B} + \frac{C(q_H + 1)^3}{2^{256}}, \end{aligned} \quad (107)$$

where $C < 648$ and $\delta_B < 2^{-227.5}$. Moreover, for any polynomial time MLWE distinguishers $\mathcal{B}_1, \mathcal{B}_2$, it holds that

$$\epsilon_{\text{CPA}, B} \leq \text{Adv}_{2,2}^{\text{MLWE}_{2,2}^{\chi_3, \chi_3}}(\mathcal{B}_1) + \text{Adv}_{3,2}^{\text{MLWE}_{3,2}^{\chi_3, \chi_2}}(\mathcal{B}_2). \quad (108)$$

PROOF. We have rewritten the domain separated Scheme B hash calls as the tagged Kyber style wrapper without changing its QROM distribution and grants the adversary the stronger full tuple oracles. Equation (88) is exactly the correctness notion required by the wrapper theorem. Applying Theorem 1 of [23] gives Eq. (107). Their constant satisfies explicitly $C < 648$. The explicit domain tags do not affect their proof because they are fixed injective encodings of the oracle roles and branch, so all queries, collisions, and reprogrammings in the wrapper proof remain well defined with the same bounds. $\square$

B.4 Threshold Security for Scheme B

We finish the security of TIND-CCA security for the TKEM scheme B by composition with privacy of the MPC layer.

THEOREM 12. *Let KEM be an IND-CCA KEM whose deterministic decapsulation function is perfectly realised by an MPC protocol satisfying the conditions in Theorem 8. Assume that decapsulation key be Shamir shared and fresh trusted preprocessing in every MPC call. Then for any TKEM adversary $\mathcal{A}$ in static semi-honest corruption of fewer than t parties, there exists KEM adversary $\mathcal{B}$ with perfect transcript simulation of the underlying KEM such that*

$$\text{Adv}_{\text{TKEM}}^{\text{TIND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}) \quad (109)$$

PROOF. The reduction is the same simulation used in Theorem 9. It samples the corrupted shares from their secret independent Shamir marginals and samples corrupted preprocessing shares from their exact subthreshold marginals. On a decapsulation query it asks the KEM decapsulation oracle for the external key and invokes the simulator of Theorem 8 to generate the corrupted protocol view.

Correctness of the MPC realisation ensures that the external key is the one prescribed by the KEM decapsulation function. The ordinary encapsulation challenge and the prohibition on challenge decapsulation are forwarded unchanged. The simulation is distributionally perfect, giving Eq. (109). $\square$

COROLLARY 13 (THRESHOLD SECURITY OF SCHEME B). *Under the two MLWE assumptions of Lemma 10, the QROM model of Theorem 11, and the trusted fresh preprocessing assumptions of Theorem 8, the threshold realisation of Scheme B is TIND-CCA secure against every static semi-honest coalition of fewer than t parties. More precisely, its TIND-CCA advantage is upper bounded by the right hand side of Eq. (107), with $\varepsilon_{\text{CPA},B}$ be bounded according to Eq. (108).*

PROOF. Recall that the threshold decapsulation protocol of scheme B computes its underlying KEM decapsulation exactly on every validated ciphertext. Applying Theorem 12 using the IND-CCA KEM bound of Theorem 11 proves the corollary. $\square$

We note that stronger security properties such as malicious security, robustness, adaptive corruption, proactive refresh, or dealerless preprocessing are not covered by our current analysis.

C Scheme A for ML-KEM-768 and 1024

Scheme A can be extended to ML-KEM-768 and 1024. The general design of MPC protocols is similar to the case of ML-KEM-512. We use the standardised parameters from FIPS 203 [26].

C.1 Scheme A for ML-KEM-768

For ML-KEM-768, the parameters are $k = 3$, $\eta_1 = \eta_2 = 2$, $d_u = 10$, and $d_v = 4$. The encapsulation key and ciphertext sizes are 1184 and 1088 bytes, respectively.

The deterministic reencryption check contains 1024 compressed coefficients. As in ML-KEM-512, each coefficient is checked using the masked compression preimage predicate of Section 5.3.2. Thus, $M_{2,\text{Eq}}^{768} = 47103$, $R_{\text{Eq}}^{768} = 16$. Encryption randomness consists of 7 PRF outputs. Since $\eta_1 = \eta_2 = 2$, every PRF call requests 128 bytes which requires one Keccak- $f[1600]$ permutation. The seven calls can be run in parallel which gives $R_{\text{PRF}}^{768} = 24$ and $M_{2,\text{PRF}}^{768} = 268800$. The seven CBD₂ outputs contain 7168 Boolean bits. Batching their conversion with the 256 decoded message bits gives $O_{2,A768} = 7424$. The fallback hash receives $z|c$ of length 1120 bytes. SHAKE256 therefore requires 9 sequential Keccak- $f[1600]$ permutations, giving $R_J^{768} = 217$ and $M_{2,J}^{768} = 345600$.

Message decoding and $G(m' || h)$ remain unchanged, with depths 6 and 24, respectively. Reencryption is publicly linear. Thus, the main validation branch has depth $R_{\text{main}}^{768} = 71$.

Since the fallback branch has depth 216, it dominates the MPC cost $R_{A,768}^{\text{dec}} = 217$. The total number of Boolean Beaver multiplications is $M_{2,A768} = 711679$. The number of arithmetic masked openings is $O_{q,A768} = 1280$. Per party communication costs is $C_{A768,\text{pp}}^{\text{out}} = 1431038\kappa + 15360$ bits.

C.2 Scheme A for ML-KEM-1024

For ML-KEM-1024, the parameters are $k = 4$, $\eta_1 = \eta_2 = 2$, $d_u = 11$, and $d_v = 5$. Both the encapsulation key and ciphertext contain 1568 bytes.

The reencryption check now covers 1280 compressed coefficients. The masked predicate itself is unchanged because coefficients are elements of $\mathbb{Z}_q$ with $q < 2^{12}$. The wider compression parameters here only change the public compression preimage intervals. Thus, $M_{2,\text{Eq}}^{1024} = 58879$ and $R_{\text{Eq}}^{1024} = 17$.

There are 9 PRF calls. Since $\eta_1 = \eta_2 = 2$, each call requires one Keccak- $f[1600]$ permutation. All nine calls can be run in parallel which gives $R_{\text{PRF}}^{1024} = 24$ and $M_{2,\text{PRF}}^{1024} = 345600$. The nine CBD₂ outputs contain 9216 bits. Including the 256 message bits, the batched conversion requires $O_{2,A1024} = 9472$.

Since the fallback input has length 1600 bytes, SHAKE256 requires 12 sequential Keccak- $f[1600]$ permutations which gives $R_J^{1024} = 288$, $M_{2,J}^{1024} = 460800$.

The ciphertext validation branch has depth $R_{\text{main}}^{1024} = 72$. This, the 288 round fallback branch again dominates which gives $R_{A,1024}^{\text{dec}} = 289$.

The total number of Boolean Beaver multiplications is $M_{2,A1024} = 915455$. The number of arithmetic masked openings is $O_{q,A1024} = 1536$. Per party communication cost is now $C_{A1024,\text{pp}}^{\text{out}} = 1840638\kappa + 18432$ bits.

D Extensions and Deployment

Our TKEM schemes are presented in the static semi-honest MPC model with trusted preprocessing and implicit rejection. We presents some extensions which are desirable for deployments with different trust or setup assumptions.

D.1 Trusted Rejector

At the output of our TKEM schemes, the parties hold shares of a secret ciphertext validity bit $[b]_2$, a secret candidate key $[K']_2$, and an implicit rejection fallback key $[K_\perp]_2$. The shares of the final session key are

$$[K]_2 = [K_\perp]_2 + [b]_2([K']_2 + [K_\perp]_2), \quad (110)$$

where addition is over $\mathbb{F}_{2^\kappa}$. This prevents any individual party from learning b .

Suppose instead that the deployment contains a dedicated Rejector $\mathcal{R}$ who is trusted to enforce the acceptance policy. The threshold parties stop after computing $([K']_2, [b]_2)$, reconstruct both values to $\mathcal{R}$ in one batched opening, and define

$$\text{RejectOut}_{\mathcal{R}}(K', b) = \begin{cases} K', & b = 1, \\ \perp, & b = 0. \end{cases} \quad (111)$$

This gives an explicit rejection functionality.

D.1.1 Rejector for Scheme A. For scheme A-512, the ciphertext validation branch ends after 95 rounds. The Rejector can remove the entire 144 round $J(z \parallel c)$ branch and the 256 bit mux. Thus, the parties can reconstruct (K', b) to the Rejector in $R_{A, \text{rej}}^{\text{out}} = 96$. One can then count the communication cost per party in this case to be $C_{A, \text{rej}}^{\text{out}} = s(714751\kappa + 12 \cdot 1024)$ bits.

D.1.2 Rejector for Scheme B. For Scheme B, the ciphertext validation branch is already deeper than either key derivation branch. Removing the bad key KT128 invocation saves one 12 round permutation's work without shortening validation. Removing the final mux saves the final core round. We still have $R_{B, \text{rej}}^{\text{out}} = 35$. Nevertheless, the per party communication cost is reduced to $C_{B, \text{rej}}^{\text{out}} = s(278527\kappa + 12 \cdot 1024)$ bits.

D.2 From Passive to Malicious MPC Security

The circuit decompositions developed in our TKEM schemes are orthogonal to the passive or active choice of MPC backend. Thus, a standard route to malicious security is to use the authenticated sharing paradigm in SPDZ [13].

Over a field $\mathbb{F}$, the parties additionally share a global MAC key α , and represent a value x by an authenticated sharing

$$\langle x \rangle_{\mathbb{F}} = ([x]_{\mathbb{F}}, [\gamma_x]_{\mathbb{F}}), \quad \gamma_x = \alpha x. \quad (112)$$

which is a relation between the reconstructed values. No individual party learns α or x . Linear operations update value and MAC shares locally. An authenticated Beaver triple $\langle a \rangle, \langle b \rangle, \langle c \rangle : c = ab$ evaluates multiplication through the same masked differences $d = x - a$, $e = y - b$, followed by the usual Beaver reconstruction of xy .

The difference from the passive backend is that opened values are accepted only if their MAC relations are checked, typically in batches using a random linear combination of the opened values. A successful forgery therefore requires violating the hidden global MAC relation. This approach is improved further in MASCOT [19] by replacing SPDZ preprocessing by an OT based protocol for generating maliciously secure multiplication material in general finite fields.

For our mixed domain computation, we can instantiate authenticated sharing independently over $\mathbb{F}_q$ and $\mathbb{F}_{2^\kappa}$. This can be done

by generating authenticated cross domain masks whose arithmetic and Boolean representations encode the same underlying value. The preprocessing phase additionally check this consistency requirement.

This extension preserves the circuit dependency graphs of MPC operations in our TKEM schemes. Overhead can be counted approximately as

$$R_{\text{mal}}^{\text{dec}} = R_{\text{core}}^{\text{dec}} + R_{\text{share-conv}} + R_{\text{MAC}}, \quad (113)$$

$$C_{\text{mal}}^{\text{dec}} = C_{\text{core}}^{\text{dec}} + C_{\text{auth}} + C_{\text{share-conv}}. \quad (114)$$

If long term inputs and all preprocessing correlations are already authenticated, then $R_{\text{share-conv}} = 0$. In this case, SPDZ style batched MAC verification only adds a constant number of synchronisation phases. Therefore, our 145 and 35 round dependency graphs of MPC operations are useful under a suitable actively secure backend.

We note that the keys in our protocols are Shamir shared with arbitrary qualified active subsets, whereas a dishonest majority SPDZ or MASCOT backend MPC engine is commonly formulated with a different authenticated sharing representation. Thus, it requires a dedicated authenticated Shamir sharing backend in implementation for this maliciously secure extensions of our TKEMs.

D.3 KEM Authenticated and Deployment

KEMTLS [28] replaces handshake signatures by a long term authentication KEM. A server side threshold committee can protect the long term decapsulation key while a frontend retains the protocol endpoint. With Scheme A, the client still sees ordinary standard ML-KEM public keys and ciphertexts. The MPC rounds are internal committee interactions, not client-server round trips.

Pre-distributed key variants of KEMTLS [29] are promising for stable enterprise provisioning. The same amortisation applies to long lived service to service channels, VPN and control plane key release or envelope encryption workflows. Our schemes might be used for these applications when the threshold servers are located at the same site, and decapsulation is not very frequent relative to the lifetime of the resulting channel.